

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS

REINFORCEMENT LEARNING IN THE DOMAIN
OF GAMES
BACHELOR'S THESIS

2025
JURAJ POVINEC

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS

REINFORCEMENT LEARNING IN THE DOMAIN
OF GAMES

BACHELOR'S THESIS

Study Programme: Applied Computer Science
Field of Study: Computer Science
Department: Department of Applied Informatics
Supervisor: doc. Ing. Peter Lacko, PhD.

Bratislava, 2025
Juraj Povinec



THESIS ASSIGNMENT

Name and Surname: Juraj Povinec
Study programme: Applied Computer Science (Single degree study, bachelor I. deg., full time form)
Field of Study: Computer Science
Type of Thesis: Bachelor's thesis
Language of Thesis: Slovak
Secondary language: English

Title: Učenie posilňovaním v doméne hier
Reinforcement learning in the domain of games

Annotation: Hlboké architektúry neurónových sietí preukázali schopnosť napodobňovať ľudí pri rôznych úlohách (rozpoznávanie obrazu, opis scény, alebo hranie hier). Veľký nárast výkonu grafických čipov nám umožňuje vytvárať a trénovať hlbšie a zložitejšie architektúry, ktoré zvyšujú úspešnosť umelých neurónových sietí. Učenie posilňovaním získava popularitu v doméne umelej inteligencie. Stavový priestor väčšiny hier je natoľko veľký, že klasické symbolické prístupy umelej inteligencie ho nedokážu efektívne prehľadávať. Preto je zaujímavé skúmať možnosť emergencie stratégie hry pri učení posilňovaním s využitím hlbokých architektúr neurónových sietí.

Aim: Preskúmajte existujúce prístupy učenia posilňovaním a vybraný prístup aplikujte na zvolenú hru. Zamerajte sa na rýchlosť učenia a kvalitu nájdenej stratégie.

Literature: Sutton, Richard S., and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 2018.

Silver, David, et al. "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play." Science 362.6419 (2018): 1140-1144.

Keywords: Učenie posilňovaním, hlboké neurónové siete, hry

Supervisor: doc. Ing. Peter Lacko, PhD.
Department: FMFI.KAI - Department of Applied Informatics
Head of department: doc. RNDr. Tatiana Jajcayová, PhD.

Assigned: 25.10.2024

Approved: 31.10.2024 doc. RNDr. Damas Gruska, PhD.
Guarantor of Study Programme

.....
Student

.....
Supervisor



ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Juraj Povinec
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Učenie posilňovaním v doméne hier
Reinforcement learning in the domain of games

Anotácia: Hlboké architektúry neurónových sietí preukázali schopnosť napodobňovať ľudí pri rôznych úlohách (rozpoznávanie obrazu, opis scény, alebo hranie hier). Veľký nárast výkonu grafických čipov nám umožňuje vytvárať a trénovať hlbšie a zložitejšie architektúry, ktoré zvyšujú úspešnosť umelých neurónových sietí. Učenie posilňovaním získava popularitu v doméne umelej inteligencie. Stavový priestor väčšiny hier je natoľko veľký, že klasické symbolické prístupy umelej inteligencie ho nedokážu efektívne prehľadávať. Preto je zaujímavé skúmať možnosť emergencie stratégie hry pri učení posilňovaním s využitím hlbokých architektúr neurónových sietí.

Cieľ: Preskúmajte existujúce prístupy učenia posilňovaním a vybraný prístup aplikujte na zvolenú hru. Zamerajte sa na rýchlosť učenia a kvalitu nájdenej stratégie.

Literatúra: Sutton, Richard S., and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 2018.

Silver, David, et al. "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play." Science 362.6419 (2018): 1140-1144.

Kľúčové slová: Učenie posilňovaním, hlboké neurónové siete, hry

Vedúci: doc. Ing. Peter Lacko, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 25.10.2024

Dátum schválenia: 31.10.2024

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Declaration of honor

I hereby declare that I have independently completed this Bachelor's thesis, titled "Reinforcement learning in the domain of games", including all appendices and figures, using the literature cited in the appended list. I declare that I have used tools of generative artificial intelligence in accordance with all relevant legal regulations, academic rights and freedoms, ethical and moral principles while upholding academic integrity, and that their use is appropriately acknowledged in the thesis.

Bratislava, 2025

.....

Juraj Povinec

Acknowledgments: I would like to thank my supervisor doc. Ing. Peter Lacko, PhD. for help and guidance while working on my Bachelor's thesis.

Abstrakt

V tejto práci sme sa venovali metódam učenia posilňovaním aplikovateľným na Atari hry, so zameraním na prostredie Space Invaders, a aplikovali sme zvolený algoritmus Soft Actor-Critic na toto prostredie. Využili sme prostredie z knižnice Arcade Learning Environment, ktorá je súčasťou rozhrania Gymnasium. Použili sme algoritmus implementovaný v knižnici CleanRL, ktorý sme následne modifikovali. Implementovali a vyhodnotili sme päť predtým navrhnutých modifikácií algoritmu, ktoré majú za cieľ zrýchliť učenie a zlepšiť výkon agenta. Naša implementácia dosiahla o 33% kratší čas tréningu a o 5% vyššie priemerné skóre ako základná verzia algoritmu.

Kľúčové slová: Učenie posilňovaním, hlboké neurónové siete, hry

Abstract

This thesis examines reinforcement learning methods applicable to Atari games, with a focus on Space Invaders, and applies the chosen discrete Soft Actor-Critic algorithm to that environment. The environment was provided by the Arcade Learning Environment framework, integrated in the Gymnasium environment interface. The algorithm was provided by the CleanRL library and was subsequently modified. We implemented and evaluated five previously proposed modifications that aim to accelerate training and improve the agent's performance. Our implementation achieved 33% faster training speed and 5% higher mean score than the base algorithm.

Keywords: Reinforcement learning, deep neural networks, games

Contents

Introduction	1
1 Reinforcement learning	3
1.1 Markov decision process	3
1.2 The exploration-exploitation trade-off	4
2 Approaches to solving Atari games	5
2.1 Human-level control through deep reinforcement learning	5
2.2 Defeating the Invaders with Deep Reinforcement Learning	6
2.3 Rainbow: Combining Improvements in Deep Reinforcement Learning .	7
2.3.1 Beyond The Rainbow: High Performance Deep Reinforcement Learning On A Desktop PC	8
2.4 Soft Actor-Critic for Discrete Action Settings	9
2.5 Summary	9
3 SAC-Discrete	11
3.1 Algorithm	11
3.1.1 Policy evaluation	12
3.1.2 Policy improvement	12
3.1.3 Temperature parameter auto tuning	13
3.2 Stored Embeddings for Efficient Reinforcement Learning	13
3.3 Stable Discrete Soft Actor-Critic	14
3.4 Munchausen Reinforcement Learning	15
3.5 SAC with Autoencoder	15
4 Atari 2600 Space Invaders	17
4.1 Overview	17
4.2 Objectives and scoring	17
4.3 Space Invaders as an environment	18
5 Technical analysis	21
5.1 Environment interfaces	21

5.1.1	Gymnasium	21
5.1.2	OpenSpiel	22
5.2	Conclusion	22
5.3	Additional libraries	23
5.3.1	Stable Baselines3	23
5.3.2	CleanRL	24
5.3.3	MLX	24
5.4	Conclusion	25
6	Implementation	27
6.1	The agent	27
6.1.1	Modifications	28
6.1.2	Miscellaneous changes	30
6.2	The environment	31
7	Results	33
7.1	Environment parallelization	33
7.1.1	One environment	34
7.1.2	Four environments	34
7.1.3	Eight environments	34
7.2	SEER	35
7.2.1	Encoder freezing	36
7.3	SAC+AE	37
7.4	SD-SAC	38
7.4.1	Tuning entropy penalty scaling factor β and clip range c	39
7.5	Munchausen Reinforcement Learning	41
7.5.1	Adjusting the clip range l_0 and scaling factor τ	41
	Conclusion	43
	Appendix A: Contents of the digital medium	49
	Appendix B: Default hyperparameters	51
	Appendix C: Evaluation results and additional training metrics	53

List of Figures

1.1	An agent interacting with its environment within an MDP	4
2.1	Architecture of the proposed DQN agent	6
3.1	Architecture of the proposed SAC+AE agent	16
4.1	Atari 2600 Space Invaders in the ALE collection of environments	18
6.1	Architecture of the agent's neural networks	28
6.2	Visualization of an observation	32
7.1	Training curves of the agents featuring one, four and eight vectorized environments	35
7.2	Training curves of various SEER agents	36
7.3	Training curves of the discrete SAC and the adapted SAC+AE agent .	38
7.4	Training curves of SD-SAC agents with various settings of the c and β parameters	40
7.5	Training curves of Munchausen RL agents with various settings of the l_0 and τ parameters	42
C.1	Q-value estimates explosion following the activation of SEER	54
C.2	Effects of hyperparameters β and c on Q-value estimation and policy entropy	55
C.3	Effects of the parameter l_0 on Q-value estimation	56
C.4	Effects of the parameter τ on Q-value estimation	57

List of Tables

7.1	Benefits in computational efficiency achieved by using multiple vector- ized environments in parallel	34
7.2	Achieved gains in training speed following the activation of SEER . . .	37
7.3	Logged training speed relevant metrics of the SAC+AE agent	38
7.4	Post-training evaluation of the SD-SAC agents	39
7.5	Post-training evaluation of the Munchausen RL agents	41
B.1	Default hyperparameter configuration	51
C.1	Evaluation results	53

Introduction

Reinforcement learning has emerged as one of the most significant paradigms in artificial intelligence. In contrast to supervised learning, where agents are provided with labeled datasets and receive explicit feedback on their predictions, reinforcement learning enables agents to discover optimal behavior through interaction with their environment. Agents receive rewards or penalties for their actions and, through trial and error, learn to optimize their strategy. This approach is particularly well suited for video games, where the state space is typically vast and dynamic, making it impractical to search through all possible states or collect comprehensive labeled training data.

Atari games have become a standard benchmark for evaluating reinforcement learning algorithms as they present various challenges including high-dimensional visual inputs that require efficient feature extraction, sparse reward signals that make learning optimal policies difficult, and the need for long-term strategic planning [18]. Despite advances in reinforcement learning, significant challenges remain in achieving efficient training and optimal performance. Algorithms often require extensive computational resources and training time to converge to satisfactory policies.

This thesis aims to enhance the performance and training speed of the selected discrete Soft Actor-Critic reinforcement learning algorithm [7] in the Space Invaders environment. We integrate the algorithm with various proposed algorithmic modifications that aim to improve the training speed and overall performance through computational efficiency, sample efficiency, and training optimization techniques.

The thesis is structured as follows. First, Chapter 1 provides an introduction to reinforcement learning. Chapter 2 explores different approaches to solving Atari games. Subsequently, Chapter 3 provides a deeper look at the selected algorithm and various proposed modifications aimed at enhancing its performance. Chapter 4 examines the Space Invaders game, our chosen experimental environment. Following this, Chapter 5 outlines the software and tools necessary to implement the agent and simulate the environment. Chapter 6 documents the implementation process, and finally, Chapter 7 presents and analyzes the results of our experiments with the proposed modifications.

Chapter 1

Reinforcement learning

We begin our thesis with a brief introduction to reinforcement learning (RL). RL is a machine learning paradigm that focuses on training an agent to make decisions through its own experience in an environment [18, 24]. An agent is the entity that learns and makes decisions about what actions to take, while the environment represents the world with which the agent interacts. The environment exists in different states (the settings of the environment) which the agent perceives through observations.

The core concept of RL involves placing an agent in an environment where it takes actions based on its current policy [18, 24]. A policy (denoted as $\pi(a \mid s)$) is the agent's strategy for selecting actions in different states. In response to these actions, the environment transitions between states and provides rewards to the agent. A reward is a numerical value that gives the agent feedback on the quality of the action it took in the given state. The agent uses these rewards to reinforce its action selection strategy and guide it toward learning the optimal policy.

The ultimate objective of the policy is to maximize long-term cumulative rewards, often referred to as the return [24]. The agent estimates these returns using value functions and derives its policy based on them. There are two main value functions, the state value function $V(s)$ and the action value function $Q(s, a)$, also known as the Q-function. While the state value function estimates the expected cumulative reward that can be achieved from the given state s under the given policy π , the Q-function estimates the expected reward of taking action a in the state s , called Q-values.

The formal framework for modeling this decision making is called the Markov decision process (MDP).

1.1 Markov decision process

The MDP is a mathematical framework that models decision making in stochastic environments [24]. An MDP is defined as a tuple $(S, A, P(s' \mid s, a), R(s, a))$, where

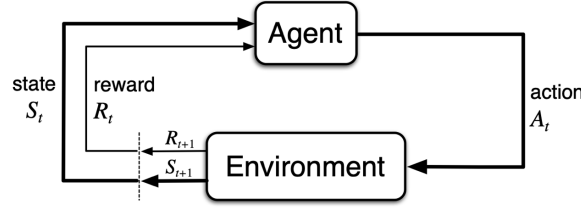


Figure 1.1: An agent interacting with its environment within an MDP [24]. Based on the observed state S_t , the agent takes an action A_t at timestep t . Subsequently, the environment reacts to the action with a reward R_{t+1} and a new state S_{t+1} .

- S is the set of all possible states in an environment.
- A is the set of all possible actions that the agent can take.
- $P(s' | s, a)$ is the transition model that defines the probability of transitioning to state s' after taking action a in state s .
- $R(s, a)$ is the reward function that outputs the immediate reward after taking action a in state s .

A crucial idea at the core of an MDP is the Markov property, which states that the future state depends only on the current state and the action taken, and not on any of the previously visited states [24]. The agent interacts with its environment in a cyclical pattern, which is illustrated in Figure 1.1.

1.2 The exploration-exploitation trade-off

A primary challenge of RL is the exploration-exploitation trade-off [24]. To achieve its goal, the agent must exploit its current optimal policy for high immediate rewards, but it must also sufficiently explore its environment to discover potentially better actions that could lead to higher returns. This forces the agent to efficiently balance between exploiting its current policy and further exploration of the environment.

To efficiently explore the environment during training, agents typically utilize the ϵ -greedy exploration strategy, where the agent takes a random action with the probability ϵ , which typically decays as training progresses [24]. Alternatively, agents employ the maximum entropy RL strategy, which encourages exploration by maximizing the entropy of the policy (preserving some randomness in the policy) in addition to cumulative rewards [18].

Chapter 2

Approaches to solving Atari games

In this chapter, we examined existing RL approaches to solving Atari games. We focused on the Space Invaders environment in particular, as we wanted to use that game to evaluate the agent. We managed to find a study that compared two different agents in the Space Invaders environment, and based on that, we searched for similar methods that we could explore.

2.1 Human-level control through deep reinforcement learning

Mnih et al. [17] proposed the Deep Q-Network (DQN), an RL method that achieves great performance in a variety of different Atari 2600 games. The DQN is a combination of deep convolutional neural networks with RL that enables the agent to learn policies directly from individual game frames. The architecture of the DQN agent can be seen in Figure 2.1.

However, the whole training process of a DQN is known to be unstable, so the authors implemented a mechanism called experience replay to remedy this issue. The experience replay is a buffer that consists of the agent's experiences stored at each time step. An experience is a tuple (*state, action, reward, next state*). The experiences stored in the experience replay were subsequently uniformly sampled during the training process. By introducing these randomly sampled experiences, the temporal correlation was broken, which led to higher stability of the training process.

Another improvement of the DQN was the addition of a target network. The target network is a clone of the Q-network that is periodically updated. Subsequently, it is frozen for the next period and used to compute the target values. Without a target network, both the predicted Q-values and the target Q-values were calculated from the same set of parameters, creating a feedback loop. This feedback loop led to instability in the training process. Using the frozen parameters of the target network, the feedback

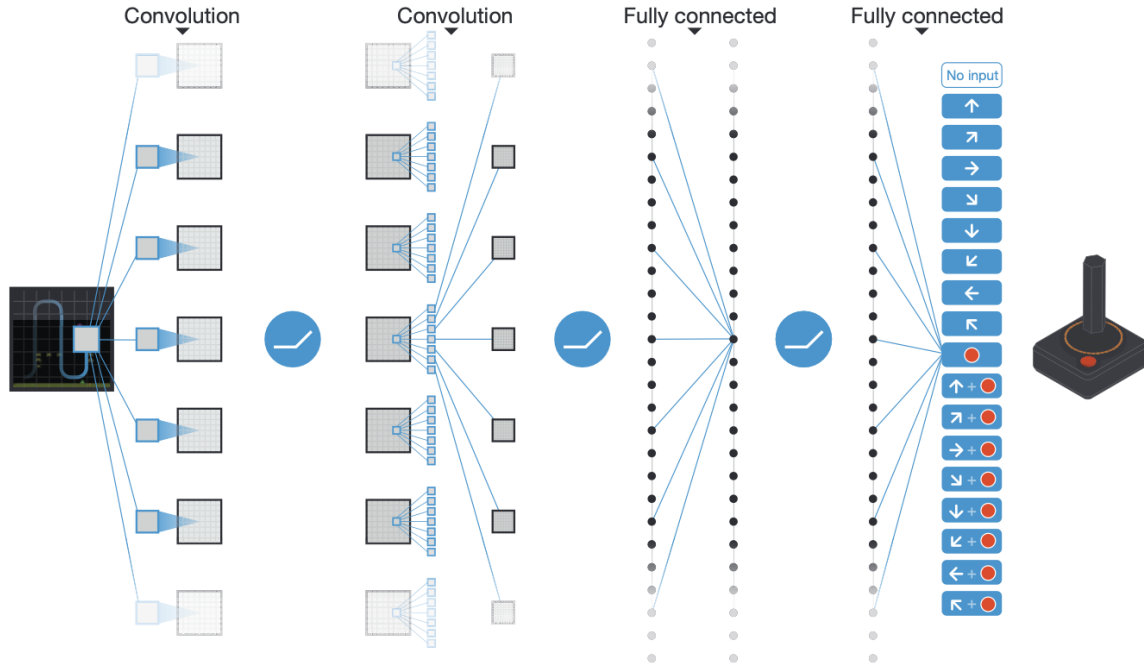


Figure 2.1: Architecture of the proposed DQN agent [17]. It consists of multiple convolutional layers that learn to correctly extract relevant features, and fully connected layers that approximate the optimal Q-function $Q(s, a)$.

loop was broken, thus improving stability and convergence.

The agent was trained on 49 different Atari 2600 games using the same hyperparameter settings, as well as the same network architecture. The agent surpassed a human game tester in performance in 29 of the 49 Atari 2600 games tested, including Space Invaders. To utilize system resources during the training process more efficiently, the input game frames were preprocessed. The preprocessing techniques used by the authors include conversion to grayscale and resolution downscaling. In addition, the input consisted of four of these game frames stacked together (frame stacking) to provide essential temporal context to the DQN.

The paper concludes with a comparison of the results of various DQN configurations, highlighting how the use of experience replay and a target network was crucial to the agent’s performance.

2.2 Defeating the Invaders with Deep Reinforcement Learning

Martinez-Nieves [16] compared the DQN agent to a different kind of RL algorithm, called the deep deterministic policy gradient (DDPG) proposed by Silver et al. [22]. The implemented DQN utilized both the experience replay and a target network; how-

ever, it had a simpler architecture compared to the DQN used by Mnih et al. [17]. The hidden convolutional layers had fewer convolutional filters, and the network had one less hidden convolutional layer overall. This naturally hindered the agent’s performance, as noted by the author in their results.

The DDPG algorithm employs a neural network architecture consisting of two neural networks, an actor, and a critic. The actor network is used to determine the policy, while the critic network approximates the Q-function. This architecture is prone to the same problems as the DQN, so both the actor and the critic networks have their respective target networks. In addition, the DDPG algorithm also utilizes experience replay to further improve stability and convergence. Furthermore, a bit of noise is added to the actor’s output to introduce some exploration.

Based on the experiments conducted by Martinez-Nieves, the DDPG agent achieved similar results in the Space Invaders environment to the DQN agent. The DDPG agent appeared to be more consistent compared to the DQN agent, while the DQN agent achieved the highest overall score during testing.

2.3 Rainbow: Combining Improvements in Deep Reinforcement Learning

Proposed by Hessel et al. [13] Rainbow is an RL algorithm that builds on the original DQN agent by combining six major extensions previously proposed for the agent to improve overall performance.

The first improvement the authors chose was the use of double Q-learning proposed by Hasselt [12], which addresses an overestimation issue in the original DQN. Instead of using a single network for both action selection and action evaluation, double q-learning employs a target network for action evaluation and the online network for action selection. This improvement is already present in the DQN proposed in the previous paper we looked at [17]; however, it was not present in the original agent.

Another improvement integrated into the Rainbow algorithm was the use of prioritized experience replay (PER) proposed by Schaul et al. [21]. Instead of uniformly sampling the replay buffer, the transitions in PER are sampled proportionally to how much the agent can still learn from them, initially favoring the most recent transitions.

The dueling network architecture proposed by Wang et al. [29] modifies the Q-network architecture to split up the Q-value estimation into two streams. This can be helpful in evaluating states where actions have little impact on the outcome.

The fourth improvement used was the use of multi-step learning proposed by Sutton [23], also called n -step learning. Instead of using only the immediate reward and the next state’s value estimate, multi-step learning calculates the target Q-value using

rewards summed over n steps with the discounted estimated value of the state reached after n steps. This modification accelerates learning.

Another modification to the neural network architecture was the introduction of noisy nets proposed by Fortunato et al. [9]. Noisy nets replace standard ϵ -greedy exploration with state-dependent exploration, which means that different states have different amounts of exploration. Noisy nets also have annealing properties, in which the agent can learn to ignore noise over time, decreasing the amount of exploration in later stages of the training process.

Finally, the last extension in Rainbow was the use of distributional Q-learning proposed by Bellemare et al. [3]. Instead of only estimating the expected return for the state-action pair (the Q-value), this approach approximates the probability distribution of possible returns.

The paper concludes by showcasing the Rainbow agent’s significant performance gain over the original DQN agent on the Atari benchmark. Furthermore, the authors compare Rainbow’s performance to a DQN agent baseline with the individual extensions. Moreover, they also provide an extensive ablation study that reveals the importance of each component, identifying PER and multi-step learning as the two most crucial components.

2.3.1 Beyond The Rainbow: High Performance Deep Reinforcement Learning On A Desktop PC

Recently, Clark et al. [8] proposed a major improvement to the Rainbow agent called Beyond the Rainbow (BTR). The BTR agent tackles poor computational efficiency of the Rainbow agent while also significantly improving its performance. The authors integrate six additional improvements into the DQN-based agent, mainly regarding its convolutional neural network (CNN) encoder. Furthermore, the agent improves Rainbow’s distributional Q-learning component and modifies estimation of target Q-values with the aim of further increasing performance. Finally, BTR utilizes environment parallelization together with a lower frequency of training updates to significantly improve training speed.

The proposed modifications significantly improved the agent’s performance, with the most crucial for overall performance being the modified CNN encoder component. However, the authors note that in earlier stages of training, other extensions were also crucial to BTR’s performance gain. Clark et al. report that the agent also achieves the desired higher computational efficiency, as the agent can be trained in one third of the time it takes to train the Rainbow agent.

2.4 Soft Actor-Critic for Discrete Action Settings

Christodoulou [7] proposed a variant of the Soft Actor-Critic (SAC) algorithm [10], which made it compatible with tasks featuring a discrete action space, such as Atari games. The SAC agent utilizes the maximum entropy RL framework, which makes the agent’s objective to maximize returns while also maximizing the policy’s entropy, effectively balancing between exploitation of the current known strategy and further exploration. SAC employs a dynamically tuned temperature parameter α that scales the entropy term in the objective function, controlling the trade-off between exploration and exploitation.

The SAC agent naturally follows the actor-critic architecture, where the actor network is used to learn the policy, while a pair of critic networks approximate the Q-function. Furthermore, the agent also uses the replay buffer used by Mnih et al. [17] in the DQN algorithm, as well as a pair of target networks.

Christodoulou proposed five modifications that made the original SAC algorithm compatible with discrete action settings. Firstly, the critic networks were changed to output Q-values for all possible actions rather than just for a given state-action pair. Secondly, the policy network was modified to directly output the probability distribution over the discrete actions instead of outputting just the mean and the covariance. Finally, the three objective functions (state value estimation, temperature value objective, and policy objective) were modified to directly use the action probability distribution.

Christodoulou evaluated the SAC-Discrete agent in selected Atari games for 100,000 steps and compared it with the Rainbow algorithm. They concluded that the agent can achieve competitive performance, as it beat the tuned Rainbow algorithm in half of the selected games despite not being fine-tuned itself.

2.5 Summary

We started by looking at the DQN agent proposed by Mnih et al. [17]. Although the agent looked promising, it was proposed in 2015, which made us believe that this agent had been thoroughly tested and improved.

Furthermore, we came across a study [16] that compared a different type of method, the DDPG algorithm, to the DQN. However, this comparison yielded mixed results as the DDPG agent performed more consistently but did not achieve overall better results than the DQN algorithm.

After looking at the DQN algorithm, we looked for improvements to the original agent and found the Rainbow algorithm [13]. Although Rainbow achieved impressive high scores on the Atari benchmark, during our research, we found that it was also

very computationally demanding. In the paper, Hessel et al. [13] state that training for 7,000,000 steps took close to 10 hours of training on a single graphics processing unit (GPU). The recently proposed BTR agent [8] attempts to remedy this issue; however, it does so at the cost of implementation complexity, integrating additional complex components.

Finally, following the comparison conducted by Martinez-Nieves [16], we looked for other actor-critic methods used in Atari games and found the SAC agent [10]. Although originally used in continuous action settings, Christodoulou [7] proposed modifications that allow it to be used in environments featuring a discrete action space. In addition, the agent achieved performance comparable to that of Rainbow while training for a limited time. However, while not explored in the paper, the architecture of the discrete SAC agent, most notably the use of two target networks, two soft Q-networks, and a policy network, suggests that it is also a computationally intensive algorithm, similar to Rainbow.

Chapter 3

SAC-Discrete

The algorithm we chose to work with was the discrete SAC algorithm [7]. We found that the agent was not as popular and, therefore, not as tested as the Rainbow algorithm [13] in discrete action settings. Furthermore, the agent outperformed Rainbow in several environments, including Space Invaders, while training for shorter periods, which was beneficial for us as we were not able to train agents for tens of millions of steps. A major downside of both Rainbow and discrete SAC is that they are both computationally demanding. We wanted to try the more efficient BTR agent [8]; however, we had problems setting it up. Moreover, due to the many different components of the BTR agent, we did not want to implement it from scratch. Additionally, we found the discrete SAC algorithm to be more comprehensible compared to Rainbow and BTR. Finally, we found various proposed extensions and improvements to the SAC agent that we could implement with the aim of boosting its performance.

In this chapter, we examined the discrete SAC agent in more depth and looked at various proposed improvements that we could integrate with the algorithm.

3.1 Algorithm

The SAC-Discrete algorithm is an off-policy algorithm, which means that the agent learns the target policy π from experiences collected using a different policy, called the behavior policy π_b [18]. This allows the agent to employ a replay buffer to store experiences in the form of $(state, action, reward, next\ state)$ tuples, as in [17], and to learn from these past experiences. Furthermore, as we mentioned in the previous chapter, the algorithm belongs to the actor-critic family of methods and its objective is to find the optimal policy π^* that maximizes the expected sum of rewards together with the policy’s entropy:

$$\pi^* = \operatorname{argmax}_{\pi} \sum_{t=0}^T \mathbb{E}_{\substack{s_t \sim S \\ a_t \sim \pi}} [\gamma(r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)))] \quad (3.1)$$

where γ is the discount factor, $r(s_t, a_t)$ is the reward after performing action a_t in state s_t at timestep t , and α is the temperature parameter that scales the entropy $\mathcal{H}$ of the policy π [7, 18, 31].

Maximizing the objective in SAC-Discrete is achieved using the soft policy iteration technique, which alternates between the policy evaluation and the policy improvement steps [7, 18, 31], as outlined in Algorithm 1.

3.1.1 Policy evaluation

During the policy evaluation step, the critics (two soft Q-function networks denoted as Q_{θ_1} and Q_{θ_2}) evaluate the expected return of the current policy π using the soft value function $V(s_t)$ defined as:

$$V(s_t) = \pi(s_t)^T [\min_{i=1,2} Q_{\theta_i}(s_t) - \alpha \ln(\pi(s_t))] \quad (3.2)$$

The critics (parameterized by θ_1 and θ_2) are updated during the policy evaluation step by minimizing the soft Bellman residual. For each critic network Q_{θ_i} , the loss function $J_Q(\theta_i)$ is:

$$J_Q(\theta_i) = \mathbb{E}_{(s_t, a_t, r(s_t, a_t), s_{t+1}) \sim D} \left[\frac{1}{2} (Q_{\theta_i}(s_t, a_t) - y(r(s_t, a_t), s_{t+1}))^2 \right] \quad (3.3)$$

where D is the replay buffer, from which transitions are sampled, and $y(r(s_t, a_t), s_{t+1})$ is the soft Q-target. The soft Q-target is estimated using the immediate reward $r(s_t, a_t)$ together with the soft value evaluation of the next state s_{t+1} under the current policy π , computed using the target critic networks parameterized by $\bar{\theta}_1$ and $\bar{\theta}_2$:

$$y(r(s_t, a_t), s_{t+1}) = r(s_t, a_t) + \gamma V_{\bar{\theta}}(s_{t+1}) \quad (3.4)$$

The target networks $Q_{\bar{\theta}_i}$ are copies of the critics networks Q_{θ_i} , which are only periodically updated using:

$$\bar{\theta}_i = \theta_i * \tau + (1 - \tau) * \bar{\theta}_i \quad (3.5)$$

where τ is a hyperparameter called the target smoothing coefficient.

3.1.2 Policy improvement

The actor (policy π_ϕ , parameterized by ϕ) is updated during the policy improvement phase. The policy parameters ϕ are updated by minimizing the policy objective $J_\pi(\phi)$:

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D} [\pi_\phi(s_t)^T [\alpha \ln(\pi_\phi(s_t)) - \min_{i=1,2} Q_{\theta_i}(s_t)]] \quad (3.6)$$

3.1.3 Temperature parameter auto tuning

Additionally, the temperature parameter α can be automatically tuned by minimizing the temperature objective $J(\alpha)$. The goal is to maintain the policy’s entropy $\mathcal{H}$ close to a preset target entropy $\bar{\mathcal{H}}$:

$$J(\alpha) = \pi_t(s_t)^T [-\alpha(\ln(\pi(s_t))) + \bar{\mathcal{H}}] \quad (3.7)$$

Algorithm 1 Discrete Soft Actor-Critic [7]

Require: Learning rates $\lambda_Q, \lambda_\pi, \lambda_\alpha$, target smoothing coefficient τ

Initialize $Q_{\theta_1} : S \rightarrow \mathbb{R}^{|A|}$, $Q_{\theta_2} : S \rightarrow \mathbb{R}^{|A|}$, $\pi_\phi : S \rightarrow [0, 1]^{|A|}$ $\triangleright$ Initialize local networks
Initialize $Q_{\bar{\theta}_1} : S \rightarrow \mathbb{R}^{|A|}$, $Q_{\bar{\theta}_2} : S \rightarrow \mathbb{R}^{|A|}$ $\triangleright$ Initialize target networks
 $\bar{\theta}_1 \leftarrow \theta_1$, $\bar{\theta}_2 \leftarrow \theta_2$ $\triangleright$ Equalize target and local network weights
 $\mathcal{D} \leftarrow \emptyset$ $\triangleright$ Initialize empty replay buffer

for each iteration **do**

for each environment step **do**

$a_t \sim \pi_\phi(a_t \mid s_t)$ $\triangleright$ Sample action from policy

$s_{t+1} \sim p(s_{t+1} \mid s_t, a_t)$ $\triangleright$ Sample transition from environment

$\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r(s_t, a_t), s_{t+1})\}$ $\triangleright$ Store transition

end for

for each gradient step **do**

$\theta_i \leftarrow \theta_{i,2} - \lambda_Q \hat{\nabla}_{\theta_{1,2}} J_Q(\theta_{1,2})$ $\triangleright$ Update Q-function parameters

$\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ $\triangleright$ Update policy parameters

$\alpha \leftarrow \alpha - \lambda_\alpha \hat{\nabla}_\alpha J(\alpha)$ $\triangleright$ Update temperature

$Q_{\bar{\theta}_{1,2}} \leftarrow \tau Q_{\theta_{1,2}} + (1 - \tau) Q_{\bar{\theta}_{1,2}}$ $\triangleright$ Update target parameters

end for

end for

return θ_1, θ_2, ϕ $\triangleright$ Optimized parameters

3.2 Stored Embeddings for Efficient Reinforcement Learning

While looking for ways to improve the computational efficiency of RL algorithms, we came across Stored Embeddings for Efficient Reinforcement Learning (SEER) [6].

Chen et al. exploit the observation that convolutional layers of encoders generally converge much faster than other parts of the agent’s networks. Using this information, the authors proposed freezing the convolutional layers of the encoder once they had converged, eliminating the need for their further training, and thus improving computational efficiency.

Another improvement proposed by Chen et al. was storing embeddings of observations instead of the observations themselves in the replay buffer. The authors argue that once the encoder is frozen, we can store embeddings, which are much smaller in size compared to observations, in the replay buffer to save memory and to dynamically increase the buffer’s size. In addition, if we utilize the storing of embeddings, further computational efficiency can be achieved because the agent samples already encoded data, completely skipping forward passes through the encoder.

Finally, the authors present their results, demonstrating that SEER dramatically improves computational efficiency without having a negative impact on the agent’s performance. Furthermore, Chen et al. mention the use of the Rainbow agent [13] as the testbed for their experiments; however, they theorize that other off-policy algorithms, such as SAC [7, 10], can also benefit from their proposed approach.

3.3 Stable Discrete Soft Actor-Critic

Recently, Zhou et al. [31] analyzed shortcomings in the adaptation of SAC to discrete action spaces [7] and proposed two modifications that make up the Stable Discrete SAC (SD-SAC) algorithm.

The first modification aimed to improve the stability of training. The authors found that the entropy of the policy falls off too early during training, causing the agent to get stuck in local optima due to insufficient exploration. Zhou et al. argue that in select environments, the agent can be deceived by short-term rewards to develop an unoptimal strategy. They propose adding an entropy penalty term to the policy’s objective function $J_\pi(\phi)$ (3.6) that aims to reduce the magnitude of change in the current policy:

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D} [\pi_\phi(s_t)^T [\alpha \ln(\pi_\phi(s_t) - \min_{i=1,2} Q_{\theta_i}(s_t))] + \beta \frac{1}{2} \mathbb{E}_{s_t \sim D} ([\mathcal{H}(\pi_{\phi_{old}}(\cdot | s_t)) - \mathcal{H}(\pi_\phi(\cdot | s_t))])^2] \quad (3.8)$$

Another shortcoming of the SAC-Discrete algorithm analyzed by Zhou et al. was the underestimation of Q-values. They argue that the underestimation bias stems from the use of double Q-learning, which was meant to combat overestimation bias. However, Zhou et al. observed that the SAC-Discrete algorithm does not tend to suffer from overestimation bias in Atari games. To mitigate the underestimation of Q-values, the authors propose modifying the soft value function $V(s_t)$ (3.2):

$$V(s_t) = \pi(s_t)^T [\text{avg}_{i=1,2} Q_{\theta_i}(s_t) - \alpha \ln(\pi(s_t))] \quad (3.9)$$

and the policy objective function $J_\pi(\phi)$ (3.6):

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D} [\pi_\phi(s_t)^T [\alpha \ln(\pi_\phi(s_t) - \text{avg}_{i=1,2} Q_{\theta_i}(s_t))] \quad (3.10)$$

to use the mean of the two Q-value estimations rather than the minimum.

The authors report that their proposed modifications improve the algorithm’s performance in most of the selected Atari games, including Space Invaders. They also noted that their changes had minimal impact on the agent’s computational efficiency, which amounted to a roughly 10% decrease in the agent’s interactions with the environment per second.

3.4 Munchausen Reinforcement Learning

We came across Munchausen RL [28] while looking at the BTR agent [8], as the algorithm also integrates this extension and found out that it can be easily incorporated into the SAC-Discrete algorithm [7].

The core idea behind Munchausen RL is to incorporate the actor’s own belief about the optimal actions directly into the reward signal it is optimizing. Vieillard et al. note that this modification had an impact in increasing the difference between optimal and suboptimal actions for the agent, making the policy more robust to approximation errors regarding the Q-values. Modifying the reward signal by adding the logarithm of the agent’s current policy in SAC-Discrete is straightforward and only requires a small modification to the calculation of the soft Q-target $y(r(s_t, a_t), s_{t+1})$ (3.4):

$$y(r(s_t, a_t), s_{t+1}) = r(s_t, a_t) + \tau [\alpha \ln \pi(a_t | s_t)]_{l_0}^0 + \gamma V_{\theta}(s_{t+1}) \quad (3.11)$$

where τ is the Munchausen term scaling factor, α is the temperature parameter, and l_0 is the Munchausen term clip range.

This simple yet effective modification achieved improved performance in 53 of the 60 games tested, where Space Invaders was among the top 10 most improved. Vieillard et al. demonstrated this modification using a modified DQN agent; however, note that this approach can be incorporated into SAC-Discrete as well.

3.5 SAC with Autoencoder

Lastly, the final extension we looked at was SAC with autoencoder (SAC+AE) proposed by Yarats et al. [30]. This modification aimed to improve sample efficiency by using a deterministic regularized autoencoder (RAE). By introducing an auxiliary image reconstruction objective, the encoder can learn to extract the relevant features from observations quicker and feed higher quality data to the actor and critic networks. This leads to faster convergence and naturally increases sample efficiency. The auxiliary image reconstruction objective $J(\text{RAE})$ introduced in the SAC+AE algorithm is defined as:

$$J(\text{RAE}) = \mathbb{E}_{o_t \sim D} [\ln p_{\theta}(o_t | z_t) + \lambda_z \|z_t\|^2 + \lambda_{\theta} \|\theta\|^2] \quad (3.12)$$

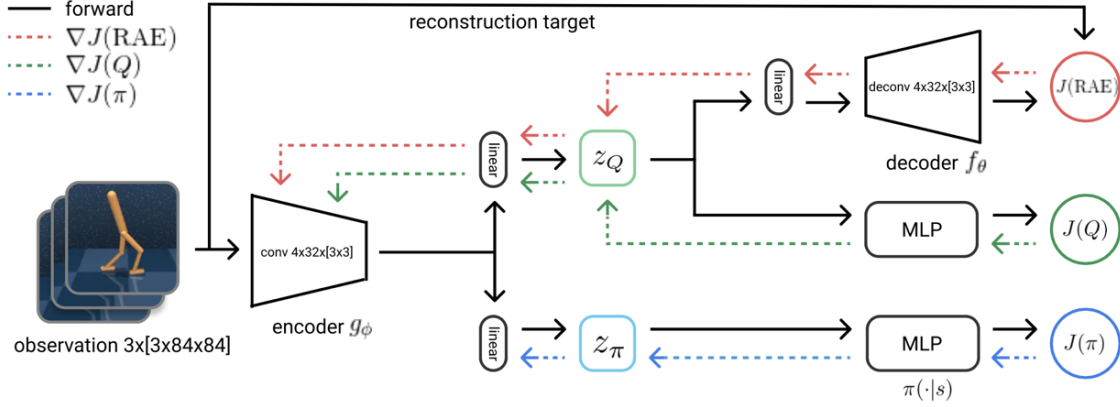


Figure 3.1: Architecture of the proposed SAC+AE agent [30]. The proposed SAC+AE agent used a single CNN encoder that was shared among the actor and critic networks. The input observation is encoded by the convolutional encoder into a 50-dimensional latent vector that is then passed to the actor and critic networks as input. In addition, the decoder attempts to reconstruct the original observation using the latent vector. The encoder is trained using gradients of the loss of this reconstruction, together with the gradients of the critics’ Q-function loss, while the decoder is trained only using the gradients of the loss of the reconstruction.

where z_t is the latent vector encoded from the observation o_t , λ_z is the L_2 penalty coefficient, and λ_θ is the weight decay coefficient for the parameters θ of the decoder network. The proposed architecture can be seen in Figure 3.1.

The SAC+AE algorithm achieved significantly better sample efficiency while preserving training stability compared to the original SAC agent [10]; however, its effectiveness is unknown in discrete action settings, as the paper only considers continuous action settings.

Chapter 4

Atari 2600 Space Invaders

Space Invaders for the Atari 2600 is a fixed shooter arcade video game released by Atari, Inc. in 1980 [1]. It is based on the original released two years prior, in 1978, by the Japanese developer Taito. There are subtle differences between these two games, mainly in the number of invading aliens, their types, and the shields available [26].

4.1 Overview

The player is tasked with defending Earth from incoming aliens, the space invaders, using a mobile laser cannon. The laser cannon can fire a laser beam directly above, as well as move horizontally along the bottom of the screen, which is useful for dodging incoming laser bombs dropped by the invading aliens. Furthermore, there are three stationary shields between the laser cannon and the space invaders, which the player can use as cover. However, these shields are destructible by both the laser cannon and the space invaders. The aliens spawn in waves of 36, six rows with six aliens in each row, and move from side to side, progressively descending to Earth. In addition, the invading aliens move increasingly faster with each successful takedown by the player. Moreover, with each wave that the player eliminates, the aliens spawn closer to Earth. Notably, after shooting down five waves, the remaining shields disappear, due to the aliens spawning so low. Occasionally, a command spaceship appears at the top of the screen, offering bonus points if it is shot down. Figure 4.1 shows the Atari 2600 Space Invaders game simulated in the Arcade Learning Environment (ALE) suite of environments [2].

4.2 Objectives and scoring

The objective of the game is to survive as long as possible, accumulating the highest possible score. The game awards the player for shooting down aliens and the command

Atari 2600 Space Invaders in the ALE collection of environments

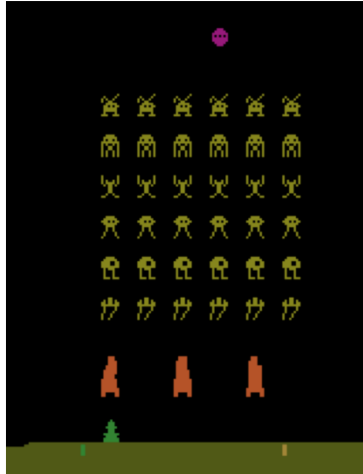


Figure 4.1: Atari 2600 Space Invaders in the ALE collection of environments [2]. This figure shows (from top to bottom) the purple command ship, 36 space invaders, the three orange shields, and the player’s laser cannon located at the bottom of the screen. Additionally, the current score is usually displayed at the top; however, it is hidden when the command ship appears.

spaceship. The command spaceship yields 200 score points while the aliens yield different amounts of score points based on their type [1]. There are a total of six types of alien, each row consisting of a single type. The aliens that spawn in the row closest to Earth yield 5 score points on takedown, the aliens in the next row are worth 10 points, up to 30 score points for aliens in the highest row. There is no penalty for being hit by the aliens; however, the game ends when the player is hit three times or when the aliens reach Earth.

In addition, there are multiple different game variations that alter the scoring system, introduce multiplayer, and change the behavior of shields, bombs, and space invaders [1].

4.3 Space Invaders as an environment

The space invaders environment requires the agent to learn effective strategies to maximize game score. Since the core gameplay loop consists of shooting down invading aliens and evading their laser bombs, the agent must learn to effectively balance offensive and defensive actions. Additionally, in the event of a command ship appearing, the agent must quickly identify and prioritize shooting it down over regular aliens to secure bonus points. Moreover, stationary shields introduce strategy to the mostly reactive

nature of the game, because they inherently slow the game down by blocking shots in both ways, requiring the agent to either wait for the aliens to pass over the shields or change targets. Most importantly, to achieve the highest scores, the agent must adapt to the changing conditions of the environment, from the shields degrading to depleting altogether, to the faster descent and lower spawn locations of the invading aliens. This combination of challenges requires effective management of the exploration-exploitation trade-off, as the agent must discover and refine strategies for balanced tactics, shield utilization, and adaptation to changing conditions. Consequently, this makes the Space Invaders environment a relevant benchmark for evaluating and experimenting with our implementation of the modified SAC agent.

Chapter 5

Technical analysis

In this chapter, we examined the key technical components of the project. First, we looked at available solutions that can provide us with Space Invaders environments to evaluate our agent. Subsequently, we looked at additional libraries that we could use to implement the chosen algorithm.

5.1 Environment interfaces

In order to train and evaluate the chosen agent, we needed an environment. During our research, we came across two major environment interfaces that provide a wide selection of environments for RL, including the aforementioned Space Invaders environments. An environment interface is a library that provides an application programming interface (API) for communication between standard RL algorithms and environments.

5.1.1 Gymnasium

Gymnasium [27] is the successor to OpenAI’s Gym [5] and is currently being maintained by The Farama Foundation. It offers a wide selection of already implemented single-agent environments. These environments range from basic ones such as Cart Pole in the Control category, to more complex environments that simulate robots physically interacting with their environment in the MuJoCo category. Additionally, Gymnasium integrates the ALE framework [2], which simulates various Atari 2600 games including Space Invaders. In addition to these environments, Gymnasium also offers numerous other external environments and supports custom environment integration using Stable-Retro [19].

Since Gymnasium is a very popular and widely used environment interface, it is also highly compatible with many libraries implementing RL algorithms. However, compatibility between Gymnasium and these frameworks varies depending on Gymnasium’s version, resulting in some frameworks using older versions.

Regarding hardware support, Gymnasium is officially available for macOS and Linux. Windows is supported, however, only unofficially.

In terms of speeding up training time, Gymnasium natively supports environment vectorization, which improves performance with little trade-off. In their paper [27], Towers et al. demonstrate how this common optimization technique increases performance on both advanced and weaker hardware setups.

5.1.2 OpenSpiel

OpenSpiel [15], developed by Google’s DeepMind, is another widely used framework that provides environments for RL. It offers a wide variety of environments including Gym’s [5] Atari environments and also implements single- and multi-agent environments. Additionally, OpenSpiel allows users to integrate custom game environments. In addition to providing environments, OpenSpiel doubles as an all-in-one framework by also including implementations of standard RL algorithms.

During our research, we could not find any popular RL frameworks that state native support for OpenSpiel, however, compatibility can be achieved using Shimmy wrappers [25]. Although these wrappers achieve compatibility, some issues can arise due to the fact that OpenSpiel does not support random seeding for all its environments, and select environments do not support graphical rendering [25]. Furthermore, since OpenSpiel relies on third-party wrappers for support for these frameworks, this solution naturally introduces a performance penalty.

OpenSpiel’s hardware compatibility presents certain limitations. Although it is officially supported on macOS and Linux with limited support for Windows 10, its support is restricted only to x86-64 based systems.

Lastly, we could not find any feature that OpenSpiel implements to help speed up training time other than the fact that the core API and games are implemented in C++ rather than Python to more effectively utilize system resources.

5.2 Conclusion

As per the environment interface, we chose to use The Farama Foundation’s Gymnasium [27]. We mainly decided to use Gymnasium because of OpenSpiel’s [15] lack of hardware support for ARM based systems. Another major reason why we chose Gymnasium over OpenSpiel is because it is supported by many frameworks that we could utilize for our project. Furthermore, Gymnasium appears to be more popular and supported in the community in general.

5.3 Additional libraries

Considering that our goal was to implement an agent, while researching additional RL libraries, we looked at how difficult it is to customize their implementations of algorithms or how easily we could implement one from scratch. We also looked at whether or not these libraries already implement the SAC [7, 10] algorithm that we could use as the basis to implement the various extensions into. Another requirement we considered was hardware support, as we would like to also use an ARM based MacBook together with a cloud workspace. Based on these requirements, we chose the following libraries to evaluate.

5.3.1 Stable Baselines3

Stable Baselines3 is a widely popular library that implements various popular RL algorithms [20].

The core features of this library are their reliable implementations of the popular RL algorithms and their simple and very user friendly API. Additionally, Stable Baselines3 provides very comprehensive documentation, together with tutorials, guides, and examples about not only the library itself but also RL in general.

Although Stable Baselines3 implements the SAC algorithm, it only implements the original agent for continuous action spaces [10]. This would require us first to adapt the agent for discrete action settings. We also looked at the more experimental repository, called SB3 Contrib¹, but could not find the implementation of the discrete SAC algorithm [7].

The algorithms implemented are customizable. Network architecture adjustments are straightforward by passing arguments during model creation. However, significant algorithmic changes require subclassing the relevant algorithm. Implementing an algorithm from scratch is also possible by inheriting from the defined base classes.

Furthermore, Stable Baselines3 also implements other components necessary for training an agent, such as preprocessing wrappers, environment wrappers, reward function wrappers, and replay buffers. We could utilize these components even if we do not choose to use Stable Baselines3 to implement our agent.

Stable Baselines3 is built on PyTorch and is designed to work with Gymnasium [27] environments. The team behind the library also offers faster, jax-based implementations².

¹<https://github.com/Stable-Baselines-Team/stable-baselines3-contrib>

²<https://github.com/araffin/sbx>

5.3.2 CleanRL

Another deep RL library we came across was CleanRL [14]. While Stable Baselines3 [20] is a fully fledged framework, CleanRL is a library that contains implementations of many popular deep RL algorithms.

A major feature of this library is that each algorithm is implemented in a single file, which means that you do not need to search across many different modules when customizing an agent. This feature makes CleanRL’s implementations easily customizable, which is perfect for us. Naturally, there is a trade-off in terms of clarity and readability because even components like the logging of the agent’s performance metrics and network definitions are contained within that single file. Some components break the single file rule and are imported, most notably the replay buffers, that are imported from Stable Baselines3. However, this does not affect the customizability in a significant way.

The library already contains a SAC implementation for both continuous and discrete action spaces [7, 10], making it a great basis for the implementation of the SAC algorithm with the various extensions.

CleanRL implements agents to work with Gymnasium [27] environments and offers implementations mainly in PyTorch; however, the versions of their dependencies are relatively old.

5.3.3 MLX

The final framework we looked at was MLX [11]. MLX is a machine learning framework similar to PyTorch and others. It is specifically designed for Apple Silicon devices and is developed by Apple machine learning research. The framework leverages a Python API similar to that of NumPy and PyTorch, making it somewhat familiar.

The main features of MLX revolve around exploiting the hardware architecture to achieve higher computational efficiency on Apple Silicon devices. However, this is also where its major drawback stems from; it only supports Apple Silicon devices.

In terms of performance, we looked at the benchmarks published by Billot [4]. In most cases, MLX is generally substantially faster than the MPS backend on PyTorch. However, one notable area where MLX is actually noticeably slower is convolutional operations. Furthermore, CUDA GPUs dramatically outperform Apple Silicon in all benchmarks done by Billot.

Since MLX is a general machine learning framework, there are obviously no preimplemented RL algorithms, meaning that we would have to implement the SAC-Discrete algorithm [7] from scratch. During our limited time testing the MLX framework, we concluded that the framework offers comprehensive support for essential machine learning components, enabling us to implement the agent from scratch.

5.4 Conclusion

Ultimately, we decided to use the CleanRL[14] implementation of the SAC-Discrete [7] algorithm to use and combine with the various extensions. We found CleanRL implementations much more accessible and easier to modify than that of Stable Baselines3 [20]. Although Stable Baselines3 allows for customization, we found it more appropriate for testing standard RL agents in unfamiliar environments, making it less ideal for our project focused on implementing algorithmic modifications. We also contemplated implementing the SAC algorithm from scratch in MLX [11]. However, we feared that the potential performance gains offered by MLX could be significantly offset by its poor performance with respect to convolutional operations. Since the agent’s architecture consists of five neural networks, each with a dedicated convolutional encoder, we concluded that it would be too computationally demanding to run the whole training process and subsequent experiments locally. Furthermore, when we tested the CleanRL implementation of the discrete SAC agent with the default hyperparameter configuration, it exceeded the performance of the same algorithm described in the SD-SAC paper [31] after 1,000,000 environmental steps. The main aspect of CleanRL that we did not like was the use of old versions of dependencies, which we planned to update when implementing the proposed improvements to the SAC agent.

Chapter 6

Implementation

For our thesis, we were tasked with implementing an RL agent of our choice and evaluating it in an environment. As we discussed in Chapter 3, we chose to implement the SAC-Discrete [7] algorithm with various extensions and evaluate it in the Space Invaders environment. The implementations can be found in our GitHub repository¹.

6.1 The agent

As the basis for our SAC agent with extensions, we used the `sac_atari.py`² agent provided by CleanRL [14]. The agent consists of two main components: the networks and the training loop.

Both the actor and the critics share the same neural network architecture. The networks consist of a CNN encoder followed by the Q-function approximator or the policy network. The CNN encoders are fed game observations in the form of pre-processed image frames, from which they extract relevant features that are subsequently used to calculate the desired output. The critics output Q-values for each action, while the actor returns logits, which are unnormalized log probabilities for each possible action. These logits are then used to get the categorical probability distribution, which is used to sample actions. The architecture of the agent’s neural networks is illustrated in Figure 6.1.

The training loop involves two main steps: data collection and agent training. Initially, during the first few thousand steps, transitions are collected by uniformly sampling the action space. The transitions are subsequently stored in the replay buffer without actual training being performed. Following the data collection period, the full training loop begins, including both gradient updates and further data collection.

¹<https://github.com/Juraj326/rl-atari/tree/main>

²https://github.com/vwxyzjn/cleanrl/blob/master/cleanrl/sac_atari.py

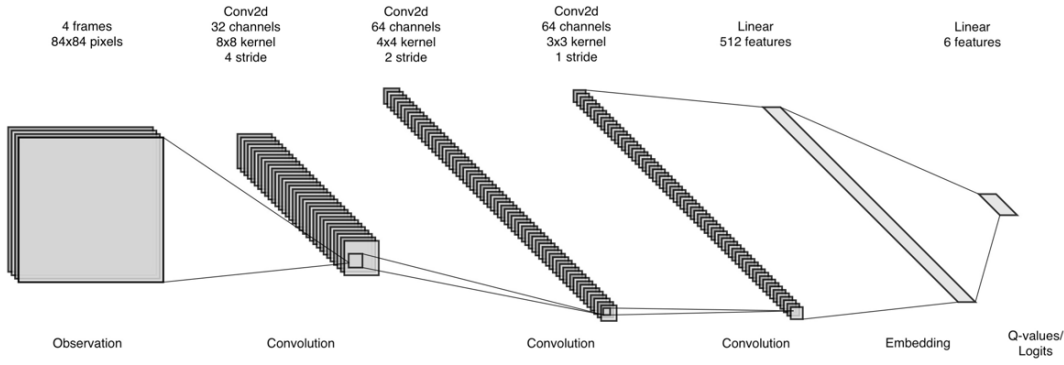


Figure 6.1: Architecture of the agent’s neural networks. The CNN encoder is made up of three convolutional `Conv2d` layers with ReLU activation functions and outputs 3136-dimensional embeddings. These embeddings are subsequently passed through two fully connected linear layers with 512 hidden neurons and the final output dimension is six, equal to the shape of the environment’s action space.

6.1.1 Modifications

Environment parallelization

As described in Chapter 5, Gymnasium [27] provides environment vectorization to accelerate training. The original agent provided by CleanRL already utilized this feature; however, we built on it and implemented environment parallelization, using asynchronous vector environments (`AsyncVectorEnv`) to further speed up training. In addition, we implemented environment parallelization while preserving the replay ratio (the ratio between environmental steps and gradient updates). For example, with a replay ratio of $\frac{1}{4}$ (one gradient update every four environmental steps), and using eight environments, the agent takes eight environmental steps in parallel, followed by two gradient updates instead of one.

SD-SAC

The second extension we implemented was the SD-SAC algorithm [31]. To implement the improvements proposed by Zhou et al., we needed to implement the two necessary components: the entropy penalty and the double average Q-learning with Q-clipping.

For the entropy penalty calculation, we first needed to modify the replay buffer to also store the entropy term. Since the agent imported the replay buffer³ from Stable Baselines3 [20], we created a new file called `memory.py`. This file only contains the relevant components of the replay buffer with the necessary modifications to work with

³https://github.com/DLR-RM/stable-baselines3/blob/master/stable_baselines3/common/buffers.py

the entropy term.

We implemented the calculation of the entropy penalty, its subsequent use in the modified policy objective function (3.8), and double average Q-learning with the Q-clipping mechanism ((3.9) and (3.10)) within the main training loop in `train.py`.

Munchausen Reinforcement Learning

Implementing the proposed changes in Munchausen RL [28] was straightforward, as we were only required to modify the reward term in the SAC target Q-value estimation formula (3.11).

SAC+AE

Furthermore, another extension that we looked at was the SAC with an autoencoder architecture as proposed by Yarats et al. [30]. Implementing this extension required a rework of both the actor and critic neural networks to be able to share an encoder, as well as the implementation of both the encoder and the decoder. In addition, we had to modify the training loop, including the addition of a section dedicated to training the autoencoder (3.12).

We used the same architecture as proposed by Yarats et al. [30]. The actor and critic networks consist of a three-layer multilayer perceptron (MLP) with 1024 hidden neurons. The encoder network consists of four convolutional layers followed by a single normalized linear layer that uses hyperbolic tangent for the activation function. All the other layers use ReLU for activation functions. The decoder mirrors the encoder network’s architecture, consisting of a fully connected linear layer followed by four deconvolutional layers.

Since the SAC+AE extension required us to make some rather extensive changes, we decided to keep it within a dedicated branch⁴.

SEER

Lastly, the final modification we wanted to implement was SEER [6]. Chen et al. proposed two key modifications: encoder freezing and storing embeddings in the replay buffer. These two modifications aim to reduce memory usage and increase the computational efficiency of the agent without hindering its performance.

Implementing embedding storage required us to modify the replay buffer to store an embedding for each of the network’s encoders excluding the target networks. This resulted in three embeddings per observation, as the actor network and the two critic networks each have their own separate encoder. Since a transition also consists of

⁴<https://github.com/Juraj326/rl-atari/tree/ae>

the next observation, the number of embeddings we needed to store per transition was six. We did not consider implementing any data augmentations for the stored embeddings, since in our case storing six embeddings per transition, in fact, required 25% more memory than storing observations. This can be seen in formula (6.1), which denotes the memory size of storing observations in relation to the memory size of storing embeddings [6]. However, we were more interested in the resources saved by not encoding observations every time they are sampled than in the memory efficiency, so we still wanted to implement this extension.

$$\frac{4 \times 84 \times 84}{4 \times 3 \times 1 \times 3136} = 0.75 \quad (6.1)$$

Rather than modifying the existing replay buffer to be compatible with both the observations and embeddings, we implemented a new replay buffer that built upon the already existing implementation and would only store the embeddings. Naturally, this required significant changes to the entire agent, since the transition to storing embeddings happens during training. However, we could not find a way to efficiently handle the transition from storing observations to embeddings, as described in the paper, by transforming the existing data in the replay buffer. Instead, we erased the existing replay buffer and initialized a new one that would store the embeddings. Following initialization, we utilized a second data collection phase before resuming training.

Implementing the encoder freezing component was substantially easier compared to storing embeddings and only required subtle changes to the encoder network architecture, as well as some utility methods.

However, due to the poor performance documented in Chapter 7, we implemented SEER only for the standard discrete SAC agent and not the SAC+AE agent.

6.1.2 Miscellaneous changes

Since the original agent required rather old versions of dependencies, we decided to update to the latest versions. Naturally, this update required us to make some changes to the codebase to be compatible. However, updating to the latest PyTorch version unlocked the ability to use the GPU of our MacBook through the Metal Performance Shaders (MPS) framework backend. This change greatly increases computational performance, enabling us to do basic debugging work locally.

In addition, with the update to the latest PyTorch version, we were able to take advantage of additional optimizations by compiling the actor, critic, and target networks. However, this feature did not benefit us in a meaningful way, since during testing, we only noticed a minimal increase in steps per second. However, it seemed to introduce instability to the training process, so we decided not to use it any further.

Furthermore, we also implemented multiple environment and reward wrappers that can be found in the `wrappers.py` file, but ultimately did not utilize them in our experiments.

Lastly, we also refactored the codebase to make it more readable and structured. Most notably, we extracted the definitions of the aforementioned critic and actor neural networks into a separate file called `networks.py`. Additionally, we extracted the CNN encoder into its own separate class to further improve the clarity of the code. Furthermore, we added comments to the training loop to more clearly denote the various sections. Finally, we moved and implemented various other utility methods into a separate file named `utils.py`.

6.2 The environment

The environment we used in this thesis is the `SpaceInvadersNoFrameskip-v4` environment provided by ALE [2]. This environment simulates the Atari2600 Space Invaders game described in Chapter 4 and is commonly used to benchmark RL algorithms.

The action space of an environment is the set of all possible actions that the agent can take. The Space Invaders environment has a discrete action space that consists of six different actions, with the possible actions being nothing, firing the cannon, moving right, moving left, moving right while firing, and moving left while firing.

The observation space of an environment is the set of all possible observations that the agent can encounter. Each observation consists of a 210×160 RGB image frame in the form of an array of the shape $(210, 160, 3)$. However, these observations are preprocessed to make training more efficient, as described in the DQN paper [17], resulting in the observation shape being $(4, 84, 84)$. The visualization of an observation can be seen in Figure 6.2.

After reviewing the implementation of the Space Invaders environment⁵, we discovered that the implemented reward function is directly tied to the scoring system. More precisely, the agent receives rewards based on the type of alien it shoots down, ranging from 5 to 30 points, just as we described in Chapter 4. However, since the original agent used reward clipping (clipping the rewards to the range $[0, 1]$), we decided to keep it.

⁵<https://github.com/Farama-Foundation/Arcade-Learning-Environment/blob/master/src/ale/games/supported/SpaceInvaders.cpp>

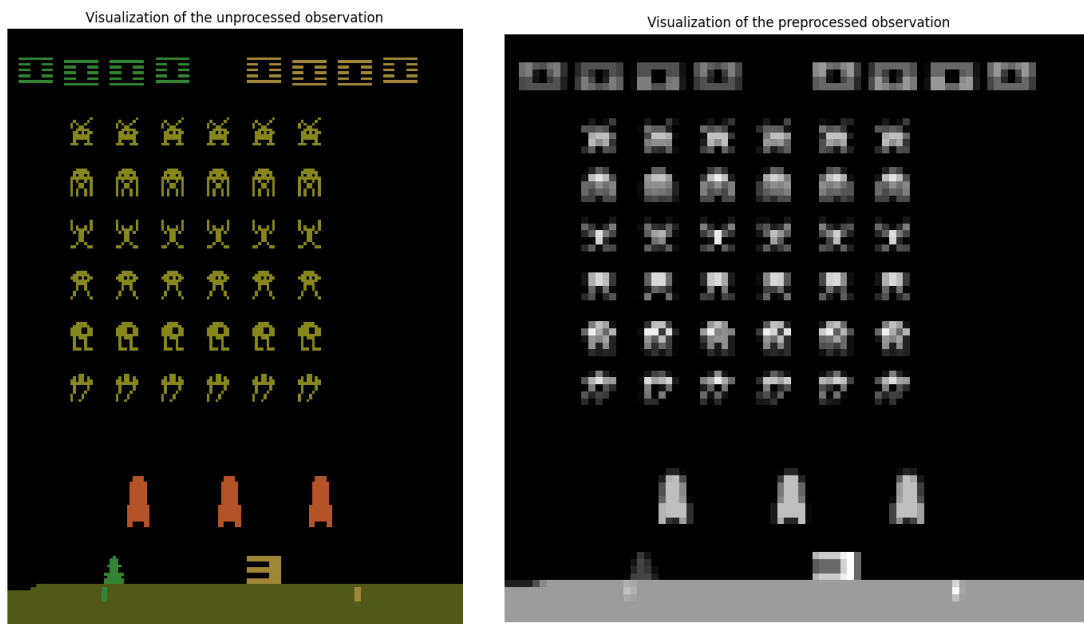


Figure 6.2: Visualization of an observation. On the left, we can see the visualization of the raw, unprocessed observation. On the right, we can see essentially what the agent sees before it gets processed by the CNN encoder. Each observation is rescaled to 84×84 , converted to grayscale, and stacked into a stack of four. Finally, the observation is normalized by 255.0 to scale each pixel value to the range $[0, 1]$.

Chapter 7

Results

In this chapter, we examined how the agent performed. We evaluated how each extension influenced the agent’s performance as well as which modifications had the most significant impact.

Each test we conducted consisted of three test runs on three random seeds. In each test, we used the SAC-Discrete agent [7] as the baseline. We used a combination of the default parameters used by the original CleanRL [14] implementation of the agent, together with the default parameters proposed in the relevant articles [6, 7, 28, 30, 31]. The default hyperparameter configuration can be seen in Table B.1 (Appendix B). All models were trained using a dedicated NVIDIA Tesla T4¹ GPU available online via the Google Colab² service. We utilized the High-RAM runtime, which featured a central processing unit (CPU) with four cores (eight with hyperthreading), and 51 GB of memory.

All models were subsequently evaluated five times on five different seeds, where each evaluation run consisted of 50 episodes. The results for all models are summarized in Table C.1, which is provided in Appendix C.

7.1 Environment parallelization

First, we looked at extensions that aim to accelerate training by improving the computational efficiency of the agent. More precisely, we first evaluated how environment parallelization influences wall time, the elapsed time in the real world that it takes for an agent to train for N environmental steps. Since the runtime we used featured four CPU cores, eight with hyperthreading, we chose to first parallelize four environments and subsequently test with eight. The test runs in this section were conducted for 2,000,000 timesteps, as we did not anticipate this modification to have any impact on

¹<https://www.nvidia.com/en-us/data-center/tesla-t4/>

²<https://colab.google>

the agent’s learning.

7.1.1 One environment

This agent acted as the baseline for this test and completed its training in an average time of 214 minutes. To gauge how fast the training process runs, we measured the agent’s steps per second, which are essentially the amount of agent’s interactions with the environment per second. The baseline agent achieved an average of 155 steps per second.

7.1.2 Four environments

Afterwards, we tested the agent with four vectorized environments running in parallel. This agent completed its training on average in 143 minutes, resulting in a roughly 33% decrease in wall time, which amounted to 71 minutes. In terms of steps per second, we recorded a significant increase to 230 steps per second, as can be seen in Table 7.1. Furthermore, the agent did not show any performance degradation, as we expected. The training curves of all the agents tested can be seen in Figure 7.1.

7.1.3 Eight environments

Lastly, we evaluated using eight vectorized environments in parallel. However, due to hardware limitations, this agent completed its training only marginally faster compared to the agent with four environments, resulting in a modest 3% decrease in wall time. On average, this agent completed its training in 137 minutes, 6 minutes faster than the four environment version, as can be seen in Table 7.1. Moreover, we observed some instability being introduced into the training process, as in one of the three test runs, the agent performed substantially worse than the baseline, as can be seen in Figure 7.1. We suspect that this instability could be caused by the eight environments steps followed by two gradient updates rather than the standard four steps with a single gradient update.

Number of environments	Wall time [minutes]	Steps per second	Mean score
1	214	155	1,039.01
4	143 (−33%)	230 (+48%)	1,021.39 (−2%)
8	137 (−36%)	240 (+55%)	954.75 (−8%)

Table 7.1: Benefits in computational efficiency achieved by using multiple vectorized environments in parallel.

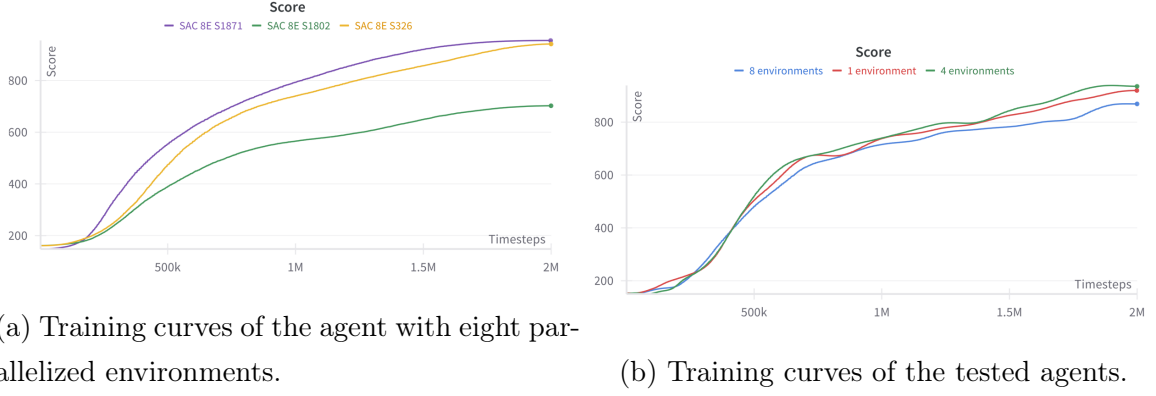


Figure 7.1: Training curves of the agents featuring one, four and eight vectorized environments.

Ultimately, four environment parallelization proved to be the most successful, as it significantly reduced wall time without compromising the agent’s performance. Therefore, we decided to utilize the four environment setup in all subsequent tests.

7.2 SEER

Following the successful integration of environment parallelization, we evaluated SEER [6]. This modification also aims to speed up training by reducing wall time while also improving memory efficiency. However, as discussed in Chapter 6, due to the architecture of our agent, storing embeddings actually required more memory. This resulted in the replay buffer shrinking from 500,000 experiences to 375,000 after the transition from storing observations to embeddings. The test runs to evaluate SEER were conducted for 5,000,000 steps to also assess its impact on the agent’s learning. As stated in the previous section, we used the SAC-Discrete agent with four parallelized environments as the baseline. The baseline completed its training for 5,000,000 steps on average in 369 minutes at 225 steps per second.

We chose to activate SEER after $T_f = 200,000$ network updates, which equals to 800,000 timesteps. Chen et al. report that freezing the encoders after 50,000 to 150,000 network updates generally performs well in Atari games; however, we chose to freeze them a bit later and then, based on the results, lower T_f to a lower value. After activating SEER, we noticed a 14% increase in steps per second compared to the baseline. This increase in the agent’s interactions with the environment per second resulted in a 12% reduction in wall time. Comparisons of steps per second after SEER activation and comparisons of wall time between agents are presented in Table 7.2. Moreover, after activating SEER, we observed that the agent’s performance degraded greatly. The performance degradation was accompanied by a massive spike in Q-value

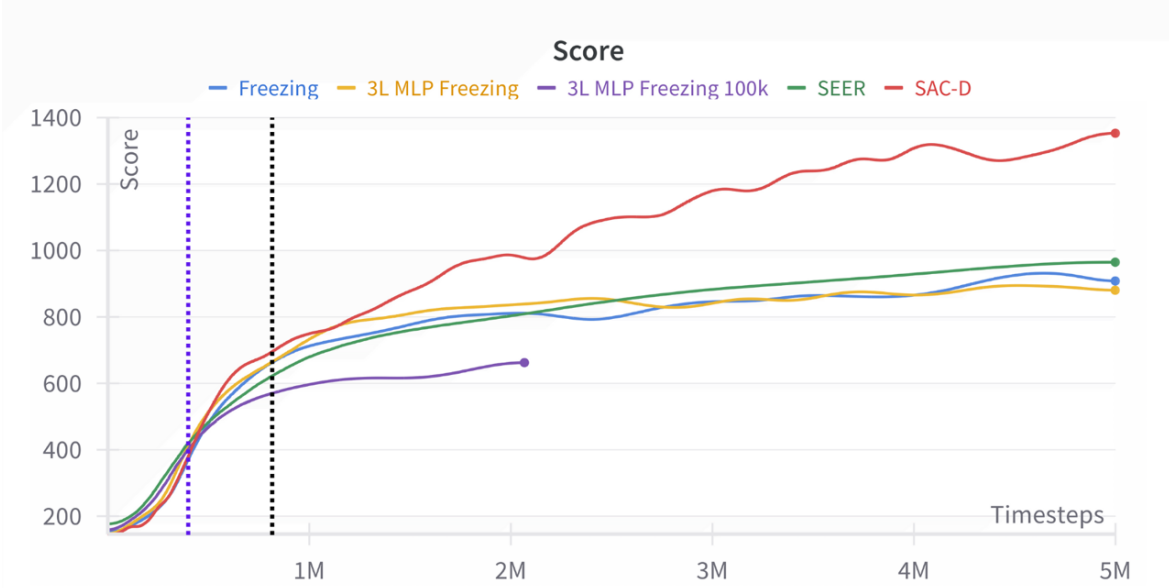


Figure 7.2: Training curves of various SEER agents. The freezing time $T_{f'} = 100,000$ is depicted by the purple dotted line while the freezing time $T_f = 200,000$ is depicted by the black dotted line.

estimates, prompting us to investigate the encoder freezing component. The training curves and the logged Q-value estimations can be seen in Figures 7.2 and C.1 (Appendix C).

7.2.1 Encoder freezing

After the first set of test runs proved unsuccessful, we evaluated only the encoder freezing component of SEER. After freezing the encoders alone, we noticed a marginal decrease of 4% in steps per second compared to SEER. This suggested that the encoders were too lightweight to see any relevant gains in training speed by bypassing their forward passes by sampling already encoded observations. However, after freezing the encoders of the individual networks, we still observed the considerable drop in the agent’s performance, resulting in the agent struggling to further optimize its policy. Figure 7.2 shows the training curves of the agents tested. Although less pronounced, following the freezing of the encoders, we observed the same explosion in the estimates of the Q-values by both critics, as can be seen in Figure C.1 (Appendix C). We suspected that this sudden increase could be attributed to the convolutional layers, not only extracting the relevant features from the observations but also contributing to estimating the Q-values.

Three-layer MLP Actor and Critics

To remedy this, we added another fully connected linear layer with 256 hidden neurons to the actor and critic networks. This resulted in the agent being marginally slower. However, as can be seen in Figures C.1 and 7.2, while this modification solved the explosion of the Q-value estimations, it failed to produce better results, as the performance of the three-layer MLP agent was comparable to that of the original.

Lowering the freezing time T_f

Finally, we tried freezing the encoders sooner, after $T_{f'} = 100,000$ network updates, as suggested in the original paper [6]. However, shortly after freezing the encoders, the agent completely stopped learning, which led us to terminate the test runs early, after only 2,000,000 timesteps.

Agent	Wall time [minutes]	Steps per second	Mean score
SAC-D	369	225	1,541.30
SEER	326 (−12%)	257 (+14%)	1,103.10 (−28%)
Freezing	333 (−10%)	248 (+10%)	998.90 (−35%)
3L MLP Freezing	347 (−6%)	237 (+5%)	1,221.81 (−21%)
3L MLP Freezing 100k	terminated	N/A	N/A

Table 7.2: Achieved gains in training speed following the activation of SEER.

Unfortunately, we were unable to successfully integrate SEER into our agent and therefore refrained from using it in subsequent evaluations.

7.3 SAC+AE

The last modification focused on accelerating training that we evaluated was SAC+AE [30]. Unlike environment parallelization and SEER, which focused on computational efficiency, SAC+AE aims to accelerate training by improving the algorithm’s sample efficiency which leads to faster convergence. Therefore, we trained the agent for 1,000,000 steps, as the agent should show an increase in performance if sample efficiency had indeed improved.

However, we found that this modification significantly underperformed compared to the baseline. Firstly, the agent was considerably slower, achieving only 56 steps per second, resulting in a wall time of 288 minutes, compared to the baseline’s 69 minutes.

In addition, the agent did not demonstrate improved sample efficiency, as its performance was worse than the baseline. We hypothesize that the autoencoder was too complex for Atari environments, causing slower learning of the relevant features. The

Agent	Wall time [minutes]	Steps per second	Mean score
SAC-D	69	155	640.69
SAC+AE	288 (+417%)	56 (−64%)	346.19 (−46%)

Table 7.3: Logged training speed relevant metrics of the SAC+AE agent.



Figure 7.3: Training curves of the discrete SAC and the adapted SAC+AE agent.

autoencoder was originally designed for continuous control tasks in the DeepMind Control Suite³, which featured 3D environments. Furthermore, hyperparameter differences also likely contributed to the agent’s poor performance, as several default parameters used by Yarats et al. differed substantially from ours, set by CleanRL [14].

However, due to the poor computational efficiency and lack of performance improvement, we decided to not experiment with SAC+AE further and leave it to future work to conserve time and computational resources in our cloud workspace for evaluating other modifications, more suitable for Atari environments.

The training curves, as well as the wall time and steps per second comparisons are presented in Table 7.3 and Figure 7.3.

7.4 SD-SAC

The first extension to the SAC-Discrete algorithm [7] that focused on the overall performance improvement we evaluated was SD-SAC [31]. For the baseline, we used the same agent as in the previous test runs, trained for 5,000,000 timesteps.

As SD-SAC modifies the exploration of the default algorithm, to make a fair com-

³https://github.com/google-deepmind/dm_control

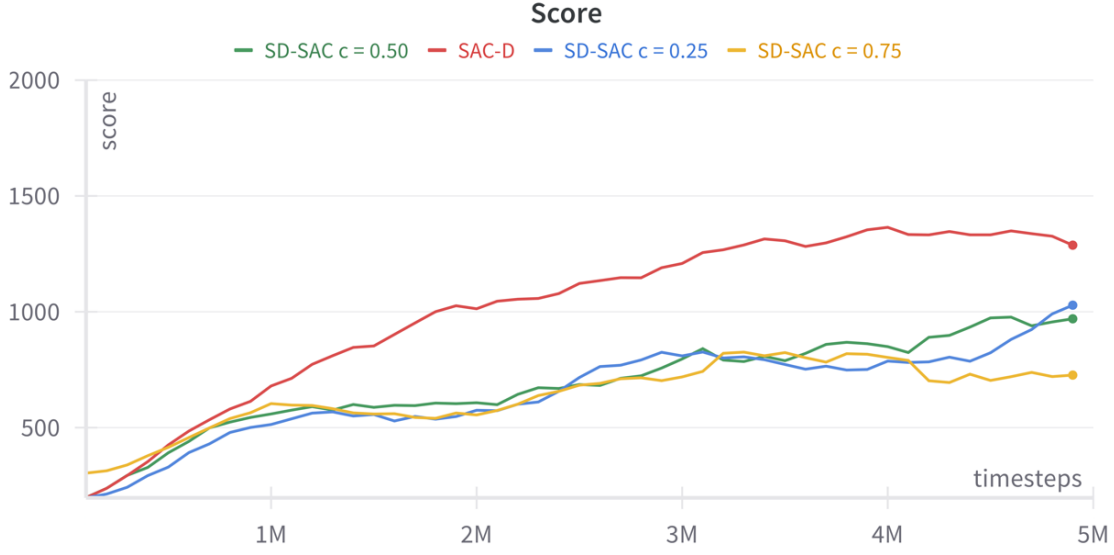
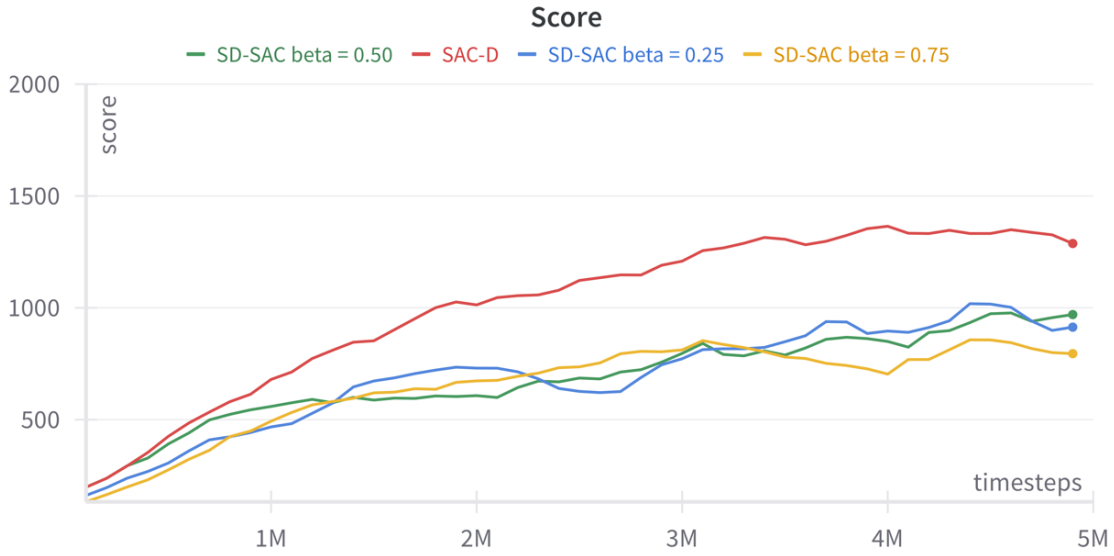
parison, we compared the performance of the deterministic policy during training. We evaluated the policy similarly as in the SD-SAC paper [31], by performing three evaluation runs with random seeds every 50,000 training steps. Each of these evaluation runs consisted of 10 episodes. During testing, we observed that the automatically tuned temperature parameter α converged to very small values, essentially 0, when the entropy penalty mechanism was used. To resolve this, we disabled the automatic tuning of α and set it to a fixed value of $\alpha = 0.05$, since Zhou et al. also used this value. First, we evaluated the SD-SAC agent with its hyperparameters β and c set to $\beta = 0.5$; $c = 0.5$, as proposed in the original paper [31]. During training, we observed only a slight 5% slowdown in steps per second. However, as can be seen in Figure 7.4 and Table 7.4, the SD-SAC agent performed worse than the vanilla discrete SAC we used as a baseline. This prompted us to experiment with the hyperparameters β and c of the SD-SAC algorithm.

7.4.1 Tuning entropy penalty scaling factor β and clip range c

To improve the performance of the SD-SAC agent we tried to both increase and lower the values of the hyperparameters β and c . However, since each experiment took approximately 20 hours to complete and due to time and resource limitations, we only examined four different parameter settings (two for each parameter). First, we adjusted the Q-clip range parameter c . Although this had little impact on the estimation of the Q-values (Figure C.2, Appendix C), we observed a notable decrease in performance. Subsequently, we tweaked the scaling factor β of the entropy penalty. This parameter had a significant impact on the policy’s entropy; however, it too failed to yield better results, and ultimately, we failed to achieve improved performance by implementing the SD-SAC agent.

Agent	Mean score
SAC-D	1,541.30
SD-SAC	1,285.62 (−17%)
SD-SAC $c = 0.75$	1,123.72 (−27%)
SD-SAC $c = 0.25$	1,155.16 (−25%)
SD-SAC $\beta = 0.75$	1,202.42 (−22%)
SD-SAC $\beta = 0.25$	1,234.31 (−20%)

Table 7.4: Post-training evaluation of the SD-SAC agents.

(a) Agents with c set to 0.25; 0.50; 0.75.(b) Agents with β set to 0.25; 0.50; 0.75.Figure 7.4: Training curves of SD-SAC agents with various settings of the c and β parameters.

7.5 Munchausen Reinforcement Learning

Finally, we evaluated the final modification we examined in this thesis; Munchausen RL [28]. Similarly to the SD-SAC agent, we observed a minor 5% drop in steps per second. Initially, we set the values of the parameters τ and l_0 to $\tau = 0.9; l_0 = -1$, just as proposed in the original paper [28] (τ is α in the paper). However, as Figure 7.5 and Table 7.5 show, this configuration performed significantly worse than the baseline discrete SAC agent. We suspected that the Munchausen term was too strong and caused the performance loss. In addition, we observed that Q-value estimates drop substantially in early stages of training (Figures C.3 and C.4, Appendix C), further signaling that the Munchausen term was too strong. To address this, we experimented with different values of the hyperparameters τ and l_0 .

7.5.1 Adjusting the clip range l_0 and scaling factor τ

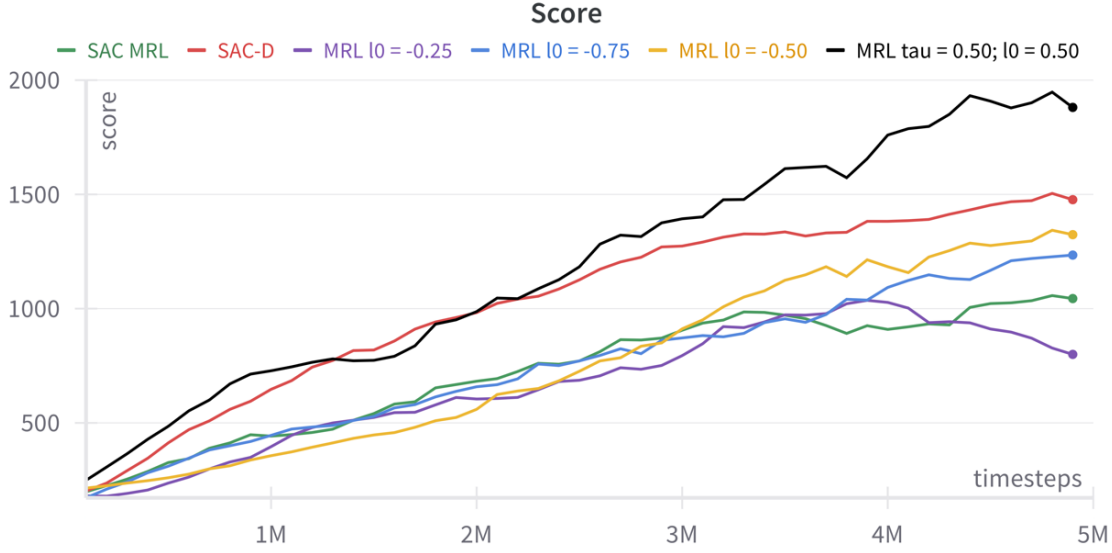
We started with lowering the clip range l_0 that bounds the values of the log-policy term, as we first wanted to decrease the influence the Munchausen term had in early stages of training, when the policy’s entropy was higher. We gradually increased l_0 from -1 to -0.25 . Although this adjustment mitigated the low Q-value estimation problem, it had a minimal effect on the agent’s performance.

Next, we tuned the parameter τ that scales the whole Munchausen term. We gradually lowered τ by three stages, from 0.9 to 0.3, which proved to be significantly more effective, as the changes to τ had a much greater impact on the agent’s performance and brought it closer to the baseline.

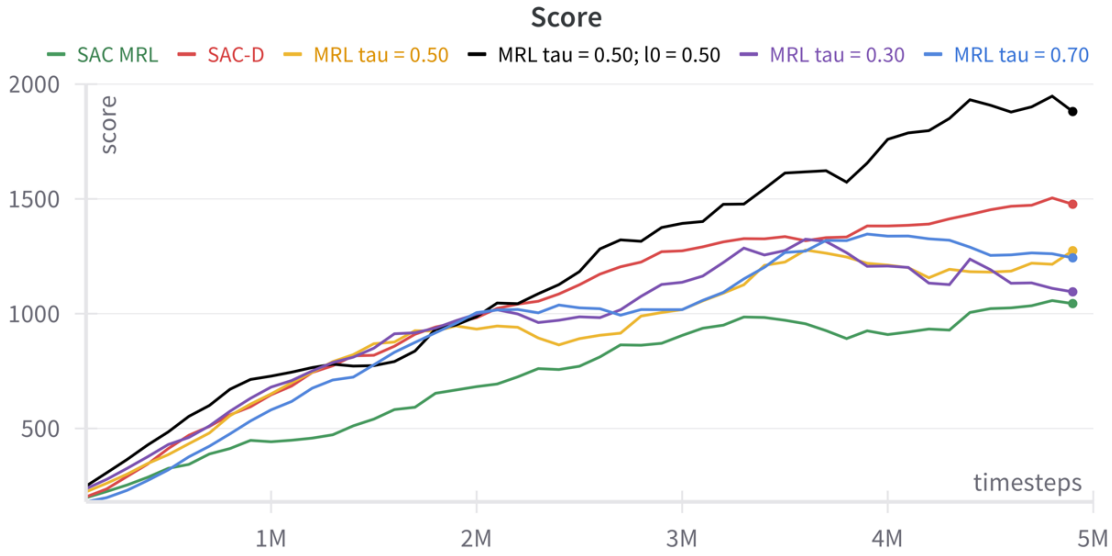
Finally, we tested one final configuration that had its parameters set to the best performing ($l_0 = -0.50$ and $\tau = 0.50$) from the previous tests. This final agent exceeded baseline performance by approximately 5% in the post-training evaluation, resulting in an average score of 1,614.08.

Agent	Mean score
SAC-D	1,541.30
SAC MRL	1,188.08 (−23%)
MRL $l_0 = -0.75$	1,145.64 (−26%)
MRL $l_0 = -0.50$	1,237.44 (−20%)
MRL $l_0 = -0.25$	1,117.15 (−28%)
MRL $\tau = 0.70$	1,406.12 (−9%)
MRL $\tau = 0.50$	1,464.12 (−5%)
MRL $\tau = 0.30$	1,439.52 (−7%)
MRL $\tau = 0.50; l_0 = -0.50$	1,614.08 (+5%)

Table 7.5: Post-training evaluation of the Munchausen RL agents.



(a) Agents with l_0 set to $-1; -0.75; -0.50; -0.25$.



(b) Agents with τ set to $0.90; 0.70; 0.50; 0.30$.

Figure 7.5: Training curves of Munchausen RL agents with various settings of the l_0 and τ parameters.

Conclusion

The aim of our thesis was to choose an environment, examine reinforcement learning methods applicable to that environment, and improve the algorithm’s training speed and performance. We chose the Atari 2600 Space Invaders environment, available in the ALE collection of environments, and the SAC-Discrete algorithm available in the CleanRL library. To improve the agent’s training speed and performance, we examined and evaluated five modifications: environment parallelization, SEER, SAC+AE, Munchausen reinforcement learning, and SD-SAC.

First, we attempted to improve the agent’s training speed by making it more computationally efficient and more sample efficient. We also adapted the agent to be compatible with the latest versions of dependencies, most notably PyTorch. This enabled us to use PyTorch’s compilation, an optimization feature, that we hoped would speed up training speed. However, this feature did not yield any positive results. Subsequently, we modified the agent to utilize multiple vectorized environments in parallel. Based on the hardware we used, we examined the parallelization of four and eight environments. The four environment setup performed the best, as it dramatically reduced wall time without compromising the agent’s performance. Afterwards, we attempted to further speed up training speed by freezing the convolutional layers of the agent’s encoders and storing embeddings rather than preprocessed observations. Although it marginally increased training speed, it also degraded the agent’s performance. We tried to mitigate this loss of performance by adding another layer to the algorithm’s policy and Q-networks; however, it was unsuccessful. Finally, we tried to make the agent more sample efficient by utilizing the SAC+AE architecture. However, the incorporation of this modification was also unsuccessful as the algorithm was significantly slower and performed substantially worse.

Second, we tried to boost the agent’s performance by implementing the SD-SAC algorithm. This method aims to improve the performance of discrete SAC by introducing an entropy penalty that gradually decreases the policy’s entropy and by modifying the target Q-value estimation to mitigate Q-value underestimation. However, even after experimenting with multiple different parameter configurations, our implementation of the SD-SAC algorithm failed to yield better results in the Space Invaders environment than the default discrete SAC agent. Finally, we evaluated the Munchausen RL agent.

At first, the agent performed significantly worse than the vanilla SAC-Discrete agent. We suspected that the performance loss was caused by the Munchausen term being too strong, as the Q-values plummeted at the start of training. After experimenting with lower values of the scaling factor τ and smaller clip ranges l_0 , we found a configuration that surpassed the baseline performance.

Ultimately, we managed to successfully improve the agent’s training speed by 33% with parallelized environments and improve the agent’s performance by 5% with the introduction of the Munchausen term to the agent’s reward signal.

The performance of the agent could be improved by successfully integrating it with the SAC+AE architecture, as the autoencoder with the auxiliary image reconstruction objective should assist it in converging faster than the standalone encoders. The computational efficiency of the agent incorporating the SAC+AE architecture could be improved by using SEER if it was also successfully integrated. The algorithm could be further improved by integrating it with other extensions such as switching out the default replay buffer for prioritized experienced replay or utilizing n -step learning.

Lastly, our post-training evaluation revealed variability in model performance, with substantial differences in mean scores achieved by different models trained with the same parameter configuration. Lowering the learning rate of the actor and critic networks or adjusting the target network smoothing coefficient τ could help mitigate this issue to produce more robust models.

Bibliography

- [1] Atari, Inc. *Space Invaders Instruction Manual*, 1980. Available at: https://atariage.com/manual_html_page.php?SoftwareLabelID=460. Accessed: 2025-04-14.
- [2] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, jun 2013.
- [3] Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International conference on machine learning*, pages 449–458. PMLR, 2017.
- [4] Tristan Billot. How fast is mlx? a comprehensive benchmark on 8 apple silicon chips and 4 cuda gpus. Towards Data Science, February 2024. Available at: <https://towardsdatascience.com/how-fast-is-mlx-a-comprehensive-benchmark-on-8-apple-silicon-chips-and-4-cuda-gpus-378a0ae356a0/>. Accessed: 2025-04-09.
- [5] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016.
- [6] Lili Chen, Kimin Lee, Aravind Srinivas, and Pieter Abbeel. Improving computational efficiency in visual reinforcement learning via stored embeddings. *Advances in Neural Information Processing Systems*, 34:26779–26791, 2021.
- [7] Petros Christodoulou. Soft actor-critic for discrete action settings. *arXiv preprint arXiv:1910.07207*, 2019.
- [8] Tyler Clark, Mark Towers, Christine Evers, and Jonathon Hare. Beyond the rainbow: High performance deep reinforcement learning on a desktop pc. *arXiv preprint arXiv:2411.03820*, 2024.
- [9] Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, et al. Noisy networks for exploration. *arXiv preprint arXiv:1706.10295*, 2017.

- [10] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.
- [11] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon, 2023.
- [12] Hado Hasselt. Double q-learning. *Advances in neural information processing systems*, 23, 2010.
- [13] Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [14] Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G.M. Araújo. Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.
- [15] Marc Lanctot, Edward Lockhart, Jean-Baptiste Lespiau, Vinicius Zambaldi, Satyaki Upadhyay, Julien Pérolat, Sriram Srinivasan, Finbarr Timbers, Karl Tuyls, Shayegan Omidshafiei, et al. Openspiel: A framework for reinforcement learning in games. *arXiv preprint arXiv:1908.09453*, 2019.
- [16] Christian L Martinez-Nieves. Defeating the invaders with deep reinforcement learning.
- [17] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- [18] Kevin Murphy. Reinforcement learning: An overview. *arXiv preprint arXiv:2412.05265*, 2024.
- [19] Mathieu Poliquin. Stable retro, a maintained fork of openai’s gym-retro. <https://github.com/Farama-Foundation/stable-retro>, 2025.
- [20] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

- [21] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.
- [22] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International conference on machine learning*, pages 387–395. Pmlr, 2014.
- [23] Richard S Sutton. Learning to predict by the methods of temporal differences. *Machine learning*, 3:9–44, 1988.
- [24] Richard S Sutton. Reinforcement learning: An introduction. *A Bradford Book*, 2018.
- [25] Jun Jet Tai, Mark Towers, and Elliot Tower. Shimmy: Gymnasium and Petting-Zoo Wrappers for Commonly Used Environments, June 2023.
- [26] The Centre for Computing History. Space invaders. <https://www.computinghistory.org.uk/det/329/Space-Invaders/>. Webpage describing the Atari 2600 game Space Invaders. Reference ID CH329. Accessed: 2025-04-14.
- [27] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [28] Nino Vieillard, Olivier Pietquin, and Matthieu Geist. Munchausen reinforcement learning. *Advances in Neural Information Processing Systems*, 33:4235–4246, 2020.
- [29] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR, 2016.
- [30] Denis Yarats, Amy Zhang, Ilya Kostrikov, Brandon Amos, Joelle Pineau, and Rob Fergus. Improving sample efficiency in model-free reinforcement learning from images. In *Proceedings of the aaai conference on artificial intelligence*, volume 35, pages 10674–10681, 2021.
- [31] Haibin Zhou, Zichuan Lin, Junyou Li, Qiang Fu, Wei Yang, and Deheng Ye. Revisiting discrete soft actor-critic. *arXiv preprint arXiv:2209.10081*, 2022.

Appendix A: Contents of the digital medium

The digital medium attached to this thesis contains the source code of all the components mentioned in Chapter 6. The source code is also published on our public GitHub repository, accessible via <https://github.com/Juraj326/rl-atari>. The digital medium has the following content:

- `main/` contains the agent with all of the modifications except SAC+AE
- `sac+ae/` contains the discrete SAC+AE agent

Both `main/` and `sac+ae/` have the following structure:

- `agent/`
 - `eval.py` Evaluation script
 - `memory.py` Modified implementation of the Stable Baselines3 replay buffer
 - `networks.py` Neural network definitions
 - `train.py` Training script
 - `utils.py` Utility method definitions
 - `wrappers.py` Environment reward wrapper definitions
- `misc/visualize_obs.py` Python script used to create Figures 4.1 and 6.2
- `README.md` Documentation with setup and usage instructions
- `requirements.txt` List of required dependencies versions

Appendix B: Default hyperparameters

The default parameters were chosen as a combination of the parameters used in the original implementation provided by [14] and the parameters proposed with the modifications we used [6, 7, 28, 30, 31].

Hyperparameter	Value
Buffer size	500,000
Discount factor γ	0.99
Target smoothing coefficient τ	1.0
Batch size	64
Interactions until training starts	20,000
Learning rate of the Actor network	0.0003
Learning rate of the Critic networks	0.0003
Network update frequency	4
Target network update frequency	8,000
Temperature coefficient α	0.05
Auto tuning of temperature coefficient α	True
Target entropy scaling factor	0.89
Number of parallel environments	4
Episodic life wrapper	True
Reward wrapper	clipped
SEER specific hyperparameters	
T_f network updates to activate SEER	200,000
SD-SAC specific hyperparameters	
Entropy penalty coefficient β	0.5
Q-clip range coefficient c	0.5
Munchausen RL specific hyperparameters	
Munchausen term scaling factor τ	0.9
Munchausen term clip range l_0	-1
SAC+AE specific hyperparameters	
Encoder learning rate	0.0003
Decoder learning rate	0.0003
Decoder weight decay coefficient λ_θ	0.0000001
L_2 penalty coefficient λ_z	0.000001

Table B.1: Default hyperparameter configuration.

Appendix C: Evaluation results and additional training metrics

This appendix presents post-training evaluation results and additional training metrics that were logged during the training of the evaluated agents.

Agent	Training length	Mean score
Random	N/A	149.65
SAC-D	1,000,000	640.69
SAC+AE	1,000,000	346.19
1 environment	2,000,000	1,039.01
4 environments	2,000,000	1,021.39
8 environments	2,000,000	954.75
SAC-D	5,000,000	1,541.30
SEER	5,000,000	1,103.10
Freezing	5,000,000	998.90
Freezing 3L MLP	5,000,000	1,221.81
3L MLP Freezing	terminated	N/A
SD-SAC	5,000,000	1,285.62
SD-SAC $c = 0.75$	5,000,000	1,123.72
SD-SAC $c = 0.25$	5,000,000	1,155.16
SD-SAC $\beta = 0.75$	5,000,000	1,202.42
SD-SAC $\beta = 0.25$	5,000,000	1,234.31
MRL	5,000,000	1,188.08
MRL $l_0 = -0.75$	5,000,000	1,145.64
MRL $l_0 = -0.50$	5,000,000	1,237.44
MRL $l_0 = -0.25$	5,000,000	1,117.15
MRL $\tau = 0.70$	5,000,000	1,406.12
MRL $\tau = 0.50$	5,000,000	1,464.12
MRL $\tau = 0.30$	5,000,000	1,439.52
MRL $\tau = 0.5, l_0 = -0.50$	5,000,000	1,614.08

Table C.1: Evaluation results. Each model was evaluated using deterministic action selection. The episodic life and reward wrappers of the environment were disabled for evaluation, and the agents were evaluated in five test runs on five different seeds with 50 episodes per test run.

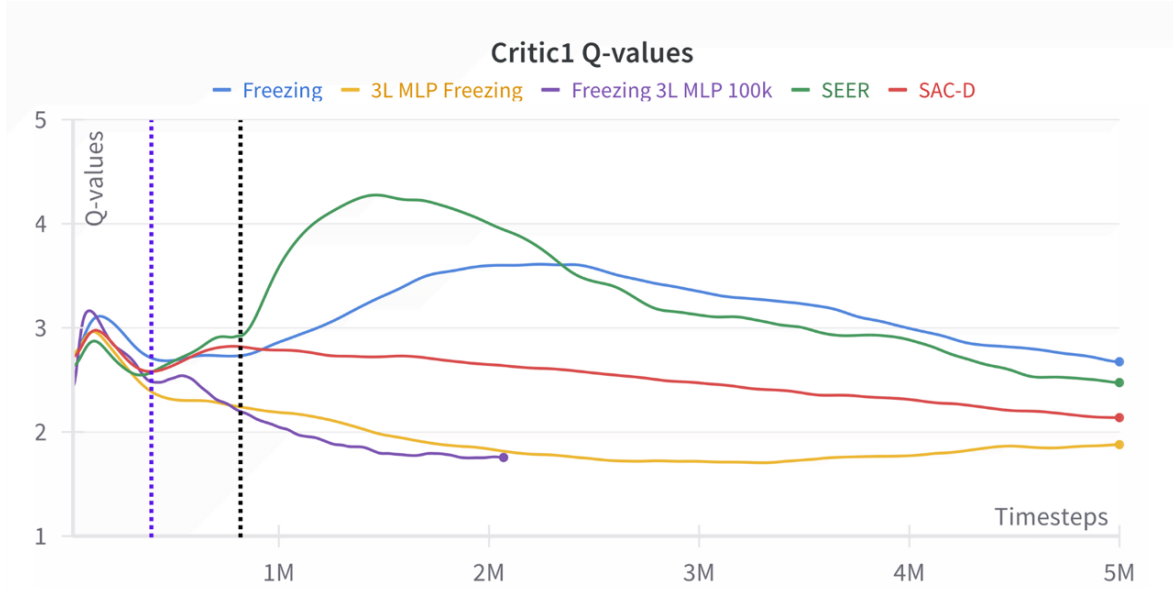
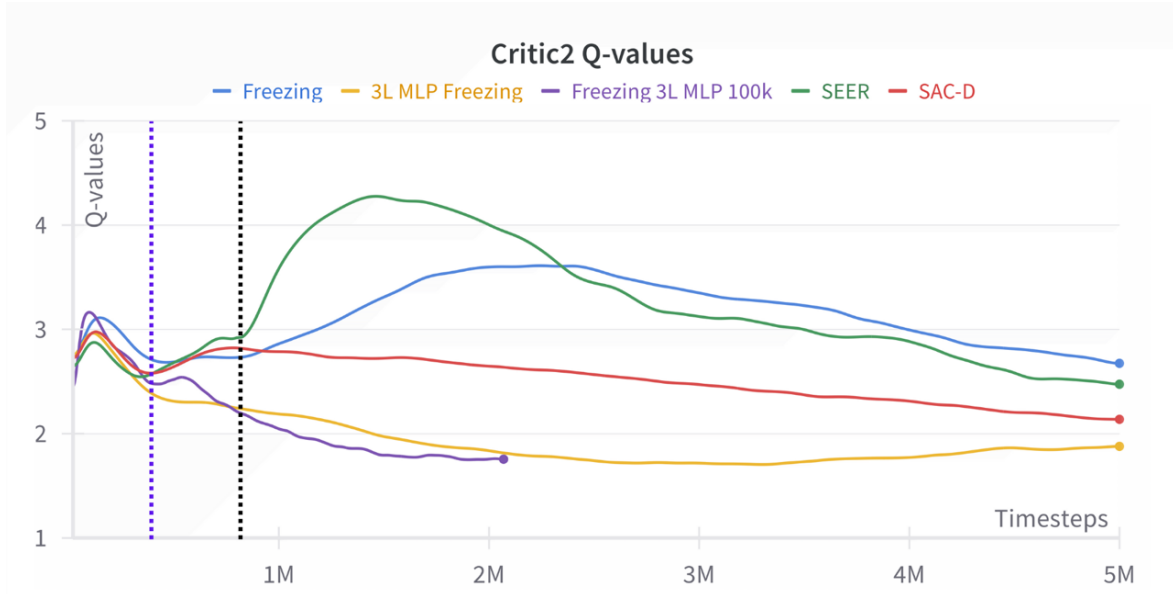
(a) Q-value estimates by Critic Q_{θ_1} (b) Q-value estimates by Critic Q_{θ_2}

Figure C.1: The logged Q-value estimates by Critics $Q_{\theta_{1,2}}$, showcasing the sudden rise in Q-value estimations following the activation of SEER and the freezing of encoders. The freezing time $T_{f'} = 100,000$ is depicted by the purple dotted line while the freezing time $T_f = 200,000$ is depicted by the black dotted line.

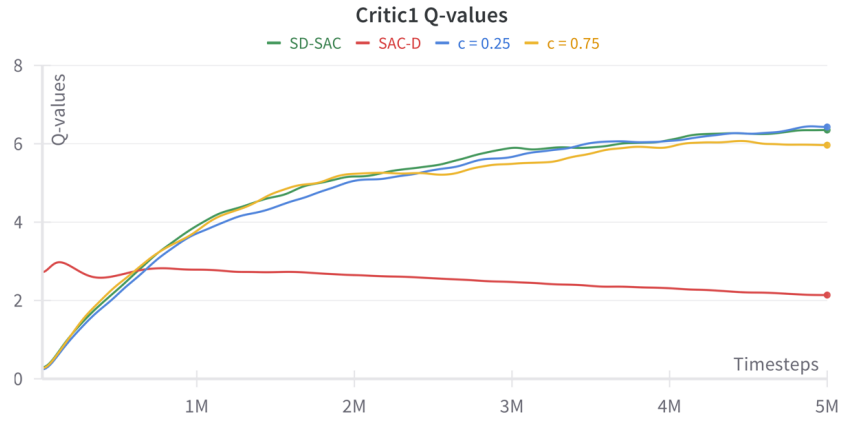
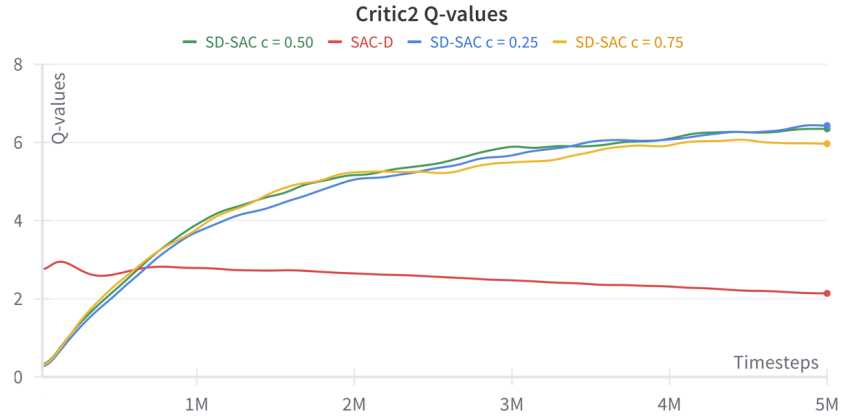
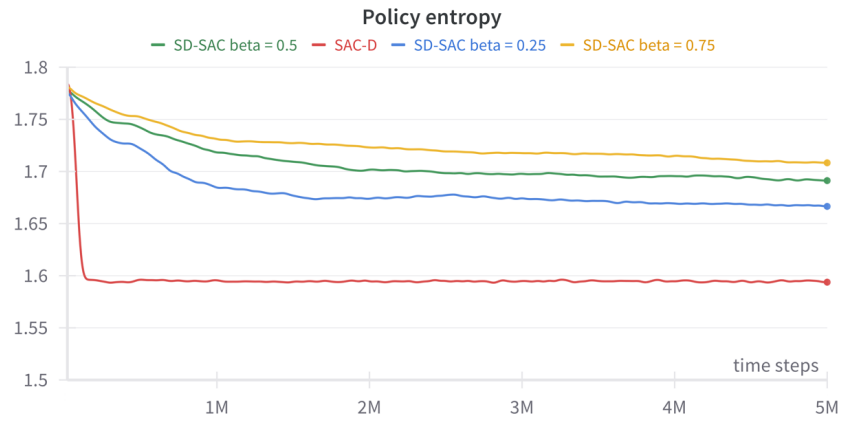
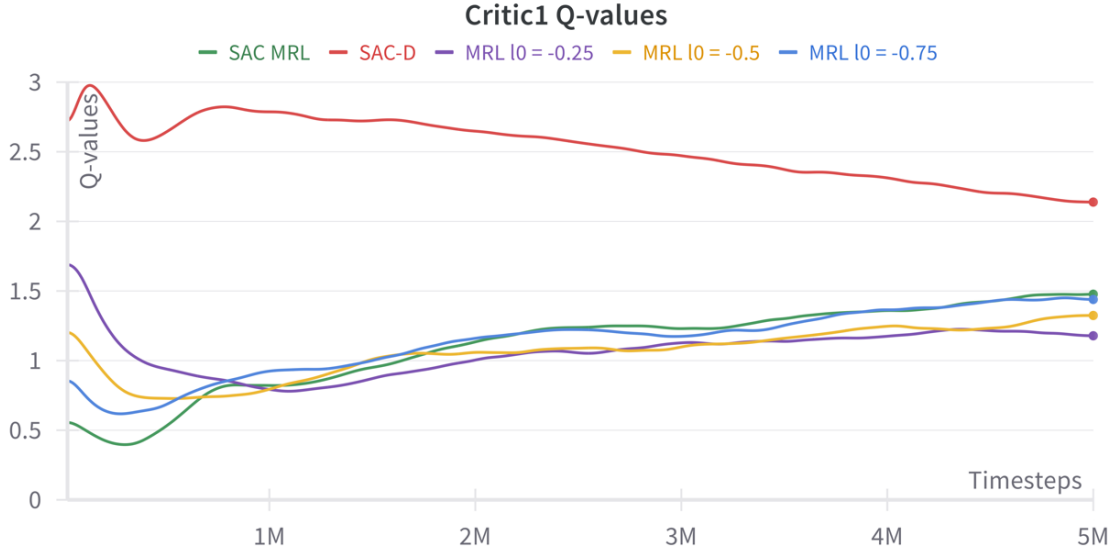
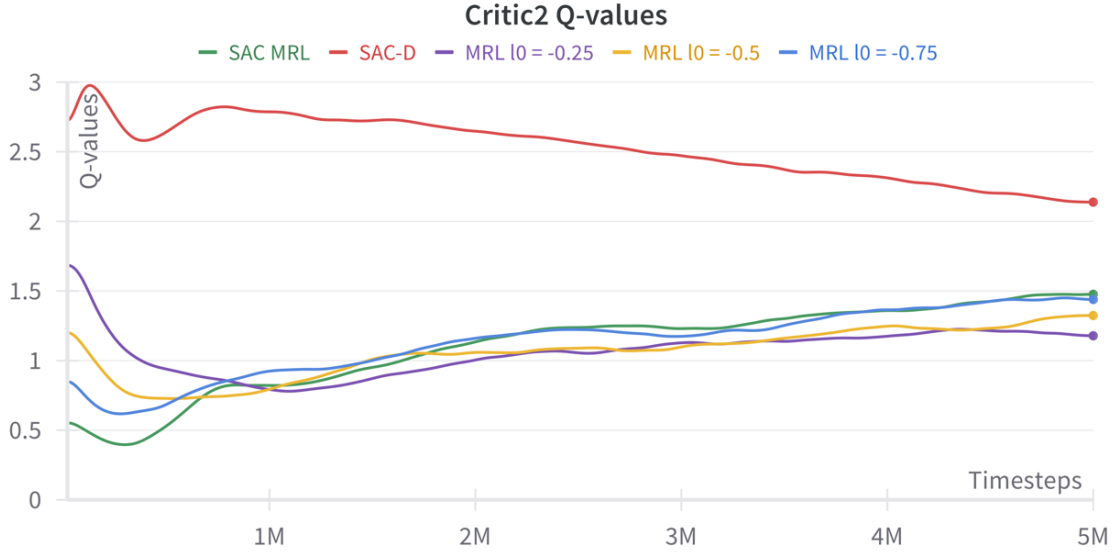
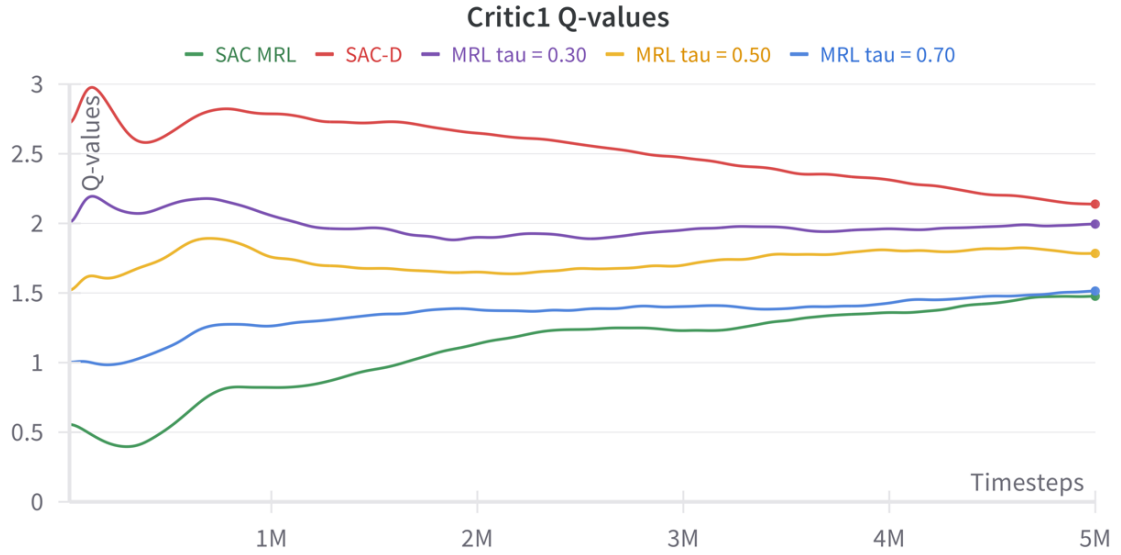
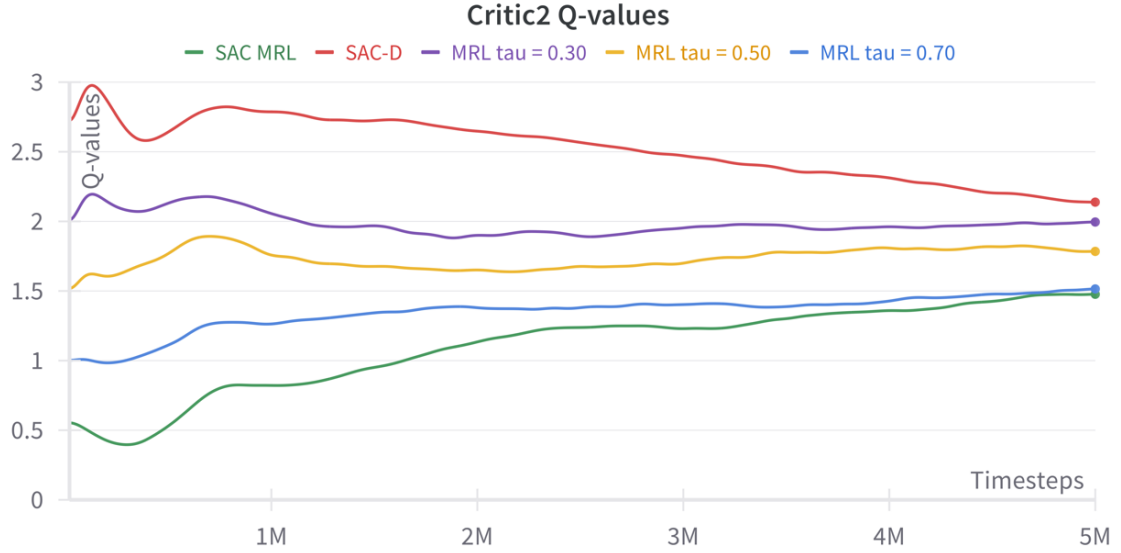
(a) Q-value estimates by Critic Q_{θ_1} (b) Q-value estimates by Critic Q_{θ_2} (c) The entropy of the current policy π_ϕ

Figure C.2: Effects of hyperparameters β and c on Q-value estimation and policy entropy. From (a) and (b), we can see that the clip range parameter c had little impact on the Q-value estimation, while the parameter β significantly influenced entropy regularization.

(a) Q-value estimates by Critic Q_{θ_1} (b) Q-value estimates by Critic Q_{θ_2} Figure C.3: Effects of the parameter l_0 on Q-value estimation.

(a) Q-value estimates by Critic Q_{θ_1} (b) Q-value estimates by Critic Q_{θ_2} Figure C.4: Effects of the parameter τ on Q-value estimation.